

STAR RX72N - rx_harq Library

1.0.0

Generated by Doxygen 1.16.1

1 Test List	1
2 Directory Hierarchy	3
2.1 Directories	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 libs/rx_harq/inc Directory Reference	7
4.2 libs Directory Reference	7
4.3 libs/rx_harq Directory Reference	8
4.4 libs/rx_harq/src Directory Reference	8
5 File Documentation	9
5.1 libs/rx_harq/inc/rx_harq.h File Reference	9
5.1.1 Detailed Description	11
5.1.1.1 HARQ Protocol Overview	11
5.1.1.2 Chase Combining Theory	11
5.1.1.3 Protocol Sequence	12
5.1.1.4 HARQ State Machine	13
5.1.1.5 Performance Characteristics	13
5.1.1.6 Features	14
5.1.2 Data Structure Documentation	15
5.1.2.1 struct rx_chase_combiner_t	15
5.1.2.2 struct rx_harq_handle_t	16
5.1.2.3 struct rx_harq_config_t	17
5.1.2.4 struct rx_harq_decode_params_t	17
5.1.3 Enumeration Type Documentation	18
5.1.3.1 rx_harq_params_t	18
5.1.3.2 rx_harq_state_t	18
5.1.4 Function Documentation	19
5.1.4.1 rx_chase_combiner_add()	19
5.1.4.2 rx_chase_combiner_can_add()	20
5.1.4.3 rx_chase_combiner_combined()	21
5.1.4.4 rx_chase_combiner_count()	22
5.1.4.5 rx_chase_combiner_deinit()	22
5.1.4.6 rx_chase_combiner_init()	23
5.1.4.7 rx_chase_combiner_reset()	24
5.1.4.8 rx_harq_can_retry()	25
5.1.4.9 rx_harq_decode()	25
5.1.4.10 rx_harq_deinit()	27
5.1.4.11 rx_harq_encode()	28
5.1.4.12 rx_harq_get_retry_count()	29

5.1.4.13 rx_harq_get_state()	30
5.1.4.14 rx_harq_init()	30
5.1.4.15 rx_harq_reset()	32
5.2 rx_harq.h	33
5.3 libs/rx_harq/src/rx_harq.c File Reference	38
5.3.1 Detailed Description	39
5.3.1.1 Implementation Architecture	39
5.3.1.2 Chase Combining Algorithm	40
5.3.1.3 Overflow Prevention	40
5.3.1.4 Protocol Flow	41
5.3.2 Enumeration Type Documentation	42
5.3.2.1 bit_constants_t	42
5.3.2.2 harq_bool_t	42
5.3.2.3 harq_cfg_sentinel_t	43
5.3.2.4 harq_fec_constants_t	43
5.3.3 Function Documentation	44
5.3.3.1 internal_handle_fec_result()	44
5.3.3.2 internal_soft_to_hard()	45
5.3.3.3 rx_chase_combiner_add()	46
5.3.3.4 rx_chase_combiner_can_add()	47
5.3.3.5 rx_chase_combiner_combined()	48
5.3.3.6 rx_chase_combiner_count()	49
5.3.3.7 rx_chase_combiner_deinit()	49
5.3.3.8 rx_chase_combiner_init()	50
5.3.3.9 rx_chase_combiner_reset()	51
5.3.3.10 rx_harq_can_retry()	52
5.3.3.11 rx_harq_decode()	52
5.3.3.12 rx_harq_deinit()	54
5.3.3.13 rx_harq_encode()	55
5.3.3.14 rx_harq_get_retry_count()	56
5.3.3.15 rx_harq_get_state()	56
5.3.3.16 rx_harq_init()	57
5.3.3.17 rx_harq_reset()	58
5.4 rx_harq.c	59

Chapter 1

Test List

File [rx_harq.h](#)

test_rx_harq.c Unit tests for HARQ protocol

Chapter 2

Directory Hierarchy

2.1 Directories

inc	7
rx_harq.h	9
libs	7
rx_harq	8
inc	7
rx_harq.h	9
src	8
rx_harq.c	38
rx_harq	8
inc	7
rx_harq.h	9
src	8
rx_harq.c	38
src	8
rx_harq.c	38

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

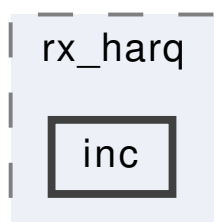
libs/rx_harq/inc/rx_harq.h	
Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N	9
libs/rx_harq/src/rx_harq.c	
Hybrid Automatic Repeat Request (HARQ) Protocol Implementation	38

Chapter 4

Directory Documentation

4.1 libs/rx_harq/inc Directory Reference

Directory dependency graph for inc:

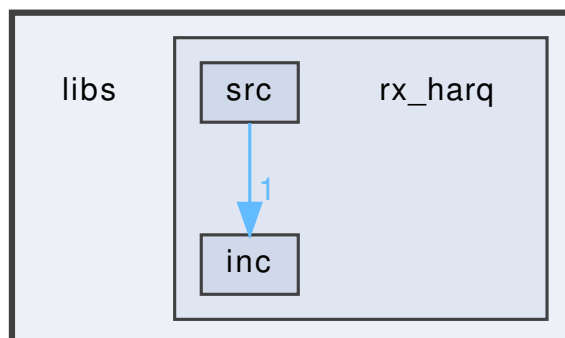


Files

- file [rx_harq.h](#)
Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

4.2 libs Directory Reference

Directory dependency graph for libs:

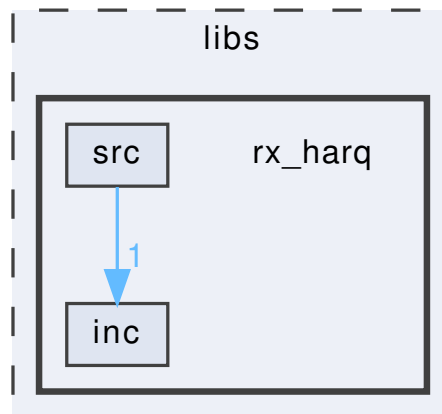


Directories

- directory [rx_harq](#)

4.3 libs/rx_harq Directory Reference

Directory dependency graph for rx_harq:

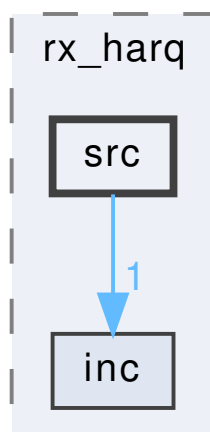


Directories

- directory [inc](#)
- directory [src](#)

4.4 libs/rx_harq/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_harq.c](#)

Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

Chapter 5

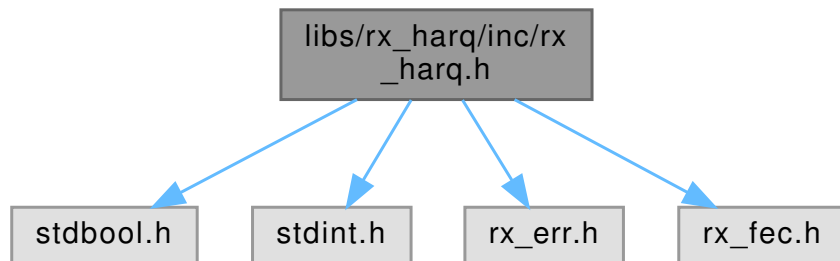
File Documentation

5.1 libs/rx_harq/inc/rx_harq.h File Reference

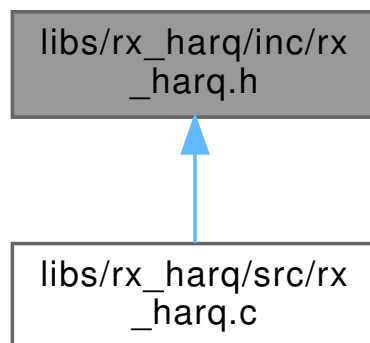
Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_fec.h"
```

Include dependency graph for rx_harq.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_chase_combiner_t](#)
Chase Combiner for soft bit accumulation.
- struct [rx_harq_handle_t](#)
HARQ protocol handle.
- struct [rx_harq_config_t](#)
HARQ configuration.
- struct [rx_harq_decode_params_t](#)
HARQ decode parameters.

Enumerations

- enum [rx_harq_params_t](#): uint16_t {
 [k_harq_max_payload](#) = 1024, [k_harq_default_retries](#) = 3, [k_harq_default_combines](#) = 3, [k_harq_soft_buffer_size](#) = 16400,
 [k_harq_survivors_buffer_size](#) = [k_harq_soft_buffer_size](#) / 2U }
HARQ protocol parameters.
- enum [rx_harq_state_t](#): uint8_t { [k_harq_state_idle](#) = 0, [k_harq_state_waiting_ack](#) = 1, [k_harq_state_combining](#) = 2, [k_harq_state_error](#) = 3 }
HARQ state machine states.

Functions

- rx_err_t [rx_chase_combiner_init](#) ([rx_chase_combiner_t](#) *combiner, uint8_t max_combines)
Initialize Chase Combiner.
- rx_err_t [rx_chase_combiner_deinit](#) ([rx_chase_combiner_t](#) *combiner)
Deinitialize Chase Combiner.
- rx_err_t [rx_chase_combiner_add](#) ([rx_chase_combiner_t](#) *combiner, const rx_soft_bit_t *soft_bits, uint32_t len)
Add soft bits from a transmission attempt.
- rx_err_t [rx_chase_combiner_combined](#) (const [rx_chase_combiner_t](#) *combiner, rx_soft_bit_t *output, uint32_t *len)
Get combined soft bits for decoding.
- rx_err_t [rx_chase_combiner_reset](#) ([rx_chase_combiner_t](#) *combiner)
Reset combiner for new frame.
- bool [rx_chase_combiner_can_add](#) (const [rx_chase_combiner_t](#) *combiner)
Check if more transmissions can be combined.
- uint8_t [rx_chase_combiner_count](#) (const [rx_chase_combiner_t](#) *combiner)
Get number of combined transmissions.
- rx_err_t [rx_harq_init](#) ([rx_harq_handle_t](#) *harq, const [rx_harq_config_t](#) *config)
Initialize HARQ handle.
- rx_err_t [rx_harq_deinit](#) ([rx_harq_handle_t](#) *harq)
Deinitialize HARQ handle.
- [rx_harq_state_t](#) [rx_harq_get_state](#) (const [rx_harq_handle_t](#) *harq)
Get current HARQ state.
- rx_err_t [rx_harq_reset](#) ([rx_harq_handle_t](#) *harq)
Reset HARQ for new transaction.
- rx_err_t [rx_harq_encode](#) (const [rx_harq_handle_t](#) *harq, const uint8_t *payload, uint32_t payload_len, uint8_t *output, uint32_t output_size, uint32_t *output_len)
Encode payload with FEC (if enabled).

- rx_err_t [rx_harq_decode](#) (rx_harq_handle_t *harq, const rx_harq_decode_params_t *params, uint8_t *output, uint32_t *output_len)
Decode soft bits with combining and FEC.
- uint8_t [rx_harq_get_retry_count](#) (const rx_harq_handle_t *harq)
Get current retry count.
- bool [rx_harq_can_retry](#) (const rx_harq_handle_t *harq)
Check if more retries are available.

5.1.1 Detailed Description

Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

Implements **Chase Combining HARQ Type I** for reliable communication over lossy channels. Combines FEC (Forward Error Correction) with ARQ (Automatic Repeat Request) to achieve both error correction and detection.

5.1.1.1 HARQ Protocol Overview

HARQ merges FEC and ARQ:

- **FEC:** Corrects random bit errors via convolutional coding
- **ARQ:** Requests retransmission when FEC fails
- **Chase Combining:** Accumulates soft bits from multiple transmissions

5.1.1.2 Chase Combining Theory

When a transmission fails to decode, instead of discarding it, the receiver:

1. **Stores** soft bits from failed attempt
2. **Requests** retransmission (NACK)
3. **Adds** new soft bits to accumulated values (element-wise)
4. **Decodes** combined soft bits (higher SNR -> better chance)

5.1.1.2.1 Mathematical Model

For n transmissions of the same data, the combined soft bit is:

$$s_{\text{combined}}[i] = \sum_{k=1}^n s_k[i]$$

This provides **diversity gain**: independent noise samples are averaged out.

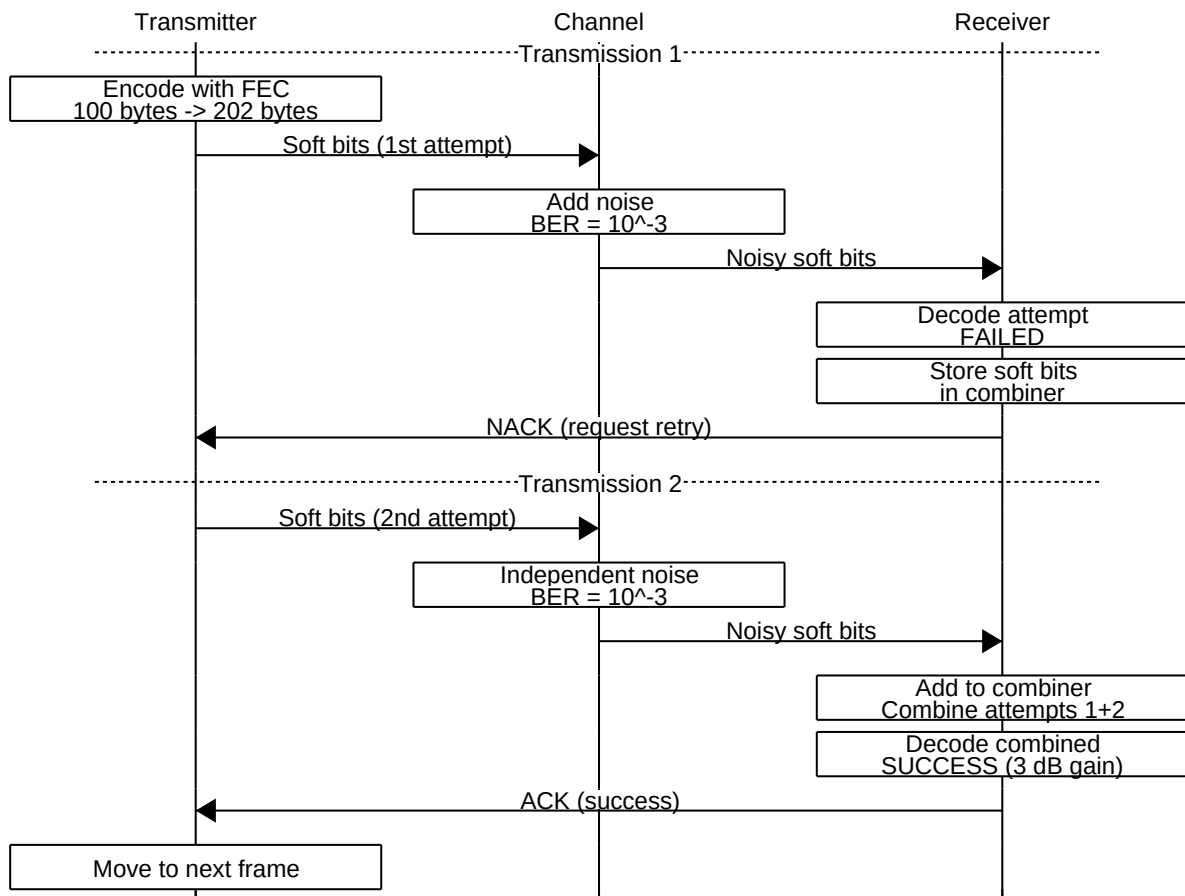
5.1.1.2.2 SNR Improvement

Assuming independent noise across transmissions, SNR improves as:

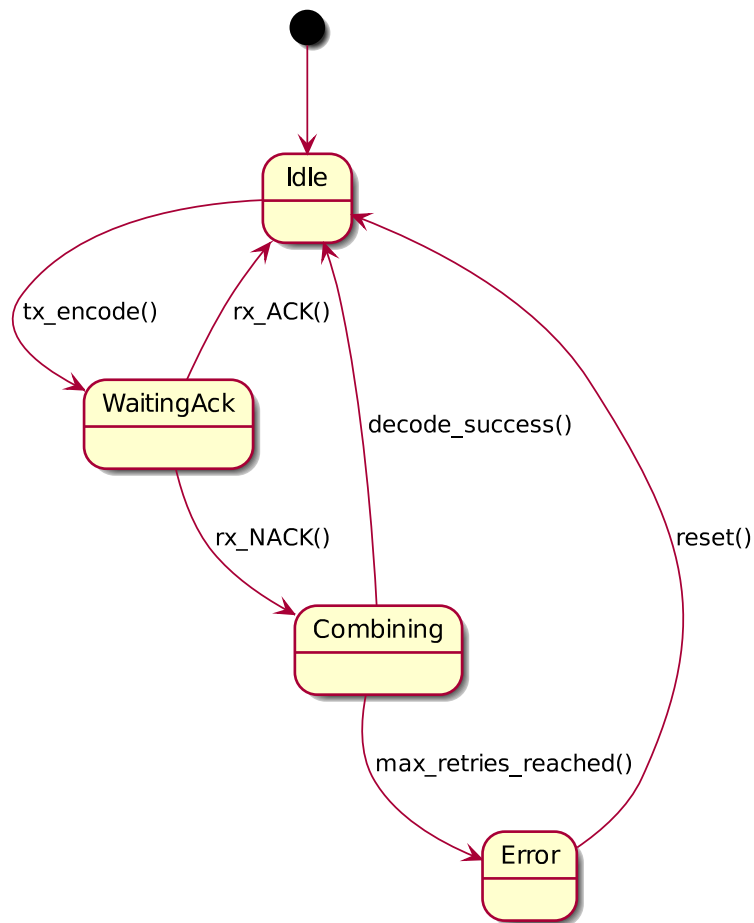
$$\text{SNR}_{\text{combined}} \approx n \cdot \text{SNR}_{\text{single}}$$

In dB: $10 \log_{10}(n)$ gain (e.g., 3 dB for 2 transmissions, 4.8 dB for 3).

5.1.1.3 Protocol Sequence



5.1.1.4 HARQ State Machine



State Descriptions:

- **Idle**: Ready for new transmission, no pending frames
- **WaitingAck**: Waiting for ACK/NACK from receiver
- **Combining**: Accumulating retransmissions, attempting decode
- **Error**: Unrecoverable error (exceeded retry limit)

Transition Guards:

- WaitingAck -> Combining: Only if `retry_count < max_retries`
- Combining -> Error: When `retry_count >= max_retries` AND decode still fails

5.1.1.5 Performance Characteristics

Metric	Value
Throughput (1 tx)	100 / 202 ~ 49.5% (FEC overhead)

Metric	Value
Throughput (2 tx avg)	100 / 404 ~ 24.8% (2x retransmissions)
Latency (1 tx)	~1 ms encode + transmit
Latency (2 tx)	~2-5 ms (includes RTT for NACK)
Memory (per handle)	~82 KB (dominated by FEC decoder)

5.1.1.6 Features

- [OK] **Chase Combining**: Soft bit accumulation (int16 to prevent overflow)
- [OK] **FEC Integration**: Optional K=7 rate 1/2 convolutional code
- [OK] **Configurable Retries**: 1-255 attempts (default: 3)
- [OK] **Static Allocation**: Zero malloc (NASA Rule 3)
- [OK] **Thread Safety**: Safe if handles not shared

Hardware Requirements

Resource	Usage
CPU	RX72N @ 240 MHz
RAM	~82 KB per HARQ handle
Flash	~8 KB (code size with FEC)

Module Dependencies

- `rx_fec.h`: FEC encoder/decoder
- `rx_err.h`: Error codes

NASA Power of 10 Compliance

- **Rule 1**: [OK] No recursion, goto, setjmp/longjmp
- **Rule 2**: [OK] All loops bounded (`k_harq_soft_buffer_size`)
- **Rule 3**: [OK] Zero dynamic allocation
- **Rule 4**: [OK] Functions ≤ 60 lines
- **Rule 5**: [OK] All functions have ≥ 2 pre/postconditions

Note

Thread Safety: HARQ handles are NOT thread-safe. Caller must provide synchronization if sharing handles between threads.

Warning

Soft bit accumulation uses int16 (range +/-32768). With max 3 combines and soft bits +/-127, accumulator saturates at +/-381 (no overflow).

See also

star-gateway/internal/harq/ Go reference implementation
 star-gateway/internal/fec/combiner.go Chase Combiner algorithm
 docs/sections/01_nanopb_protocol.tex Protocol specification
 "Hybrid ARQ Schemes" by S. Lin & D. Costello

Author

STAR Team

Date

2026-01-27

Copyright

Copyright (c) 2026 STAR Project. Licensed under MIT License.

Since

Version 1.0.0

Version

1.0.0

Test test_rx_harq.c Unit tests for HARQ protocol

Definition in file [rx_harq.h](#).

5.1.2 Data Structure Documentation

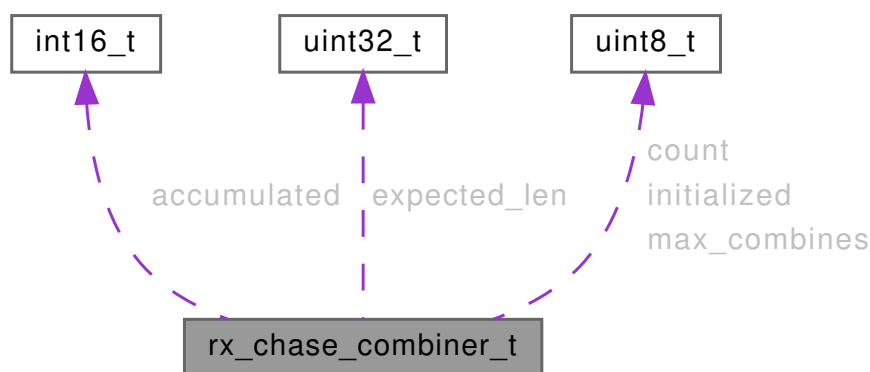
5.1.2.1 struct rx_chase_combiner_t

Chase Combiner for soft bit accumulation.

Stores soft bits from failed transmission attempts and combines them element-wise (addition) before decoding. Uses int16 accumulators to prevent overflow.

Definition at line 204 of file [rx_harq.h](#).

Collaboration diagram for rx_chase_combiner_t:



Data Fields

int16_t	accumulated[k_harq_soft_buffer_size]	Accumulated soft bits
---------	--	-----------------------

uint8_t	count	Transmissions combined
uint32_t	expected_len	Expected soft bits len
uint8_t	initialized	Non-zero if initialized
uint8_t	max_combines	Max combining attempts

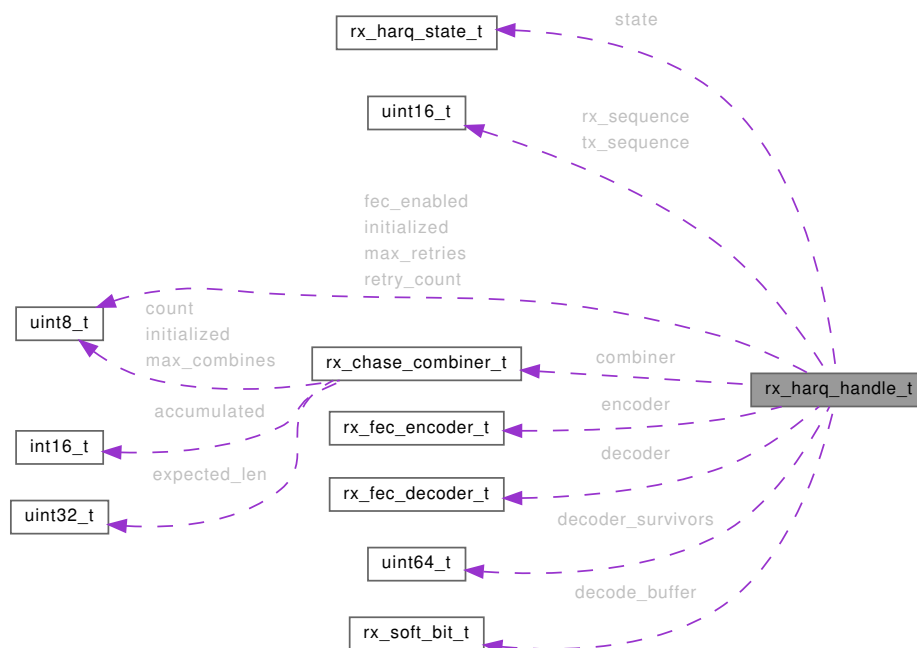
5.1.2.2 struct rx_harq_handle_t

HARQ protocol handle.

Contains all state for HARQ operation including FEC encoder/decoder and Chase Combiner.

Definition at line 303 of file [rx_harq.h](#).

Collaboration diagram for rx_harq_handle_t:



Data Fields

rx_chase_combiner_t	combiner	Chase Combiner
rx_soft_bit_t	decode_buffer[k_harq_soft_buffer_size]	Combined soft bits buf
rx_fec_decoder_t	decoder	FEC decoder
uint64_t	decoder_survivors[k_harq_survivors_buffer_size]	Decoder buf
rx_fec_encoder_t	encoder	FEC encoder
uint8_t	fec_enabled	Non-zero if FEC is enabled
uint8_t	initialized	Non-zero if initialized
uint8_t	max_retries	Maximum retries allowed
uint8_t	retry_count	Current retry count

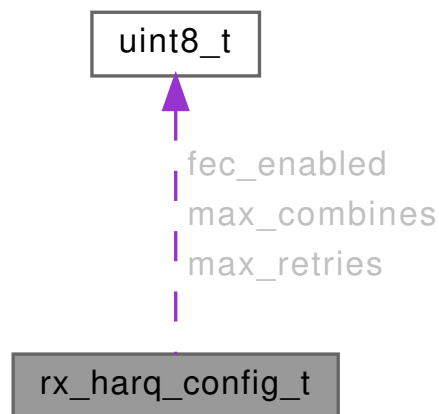
uint16_t	rx_sequence	RX sequence number
rx_harq_state_t	state	Current HARQ state
uint16_t	tx_sequence	TX sequence number

5.1.2.3 struct rx_harq_config_t

HARQ configuration.

Definition at line 321 of file [rx_harq.h](#).

Collaboration diagram for rx_harq_config_t:



Data Fields

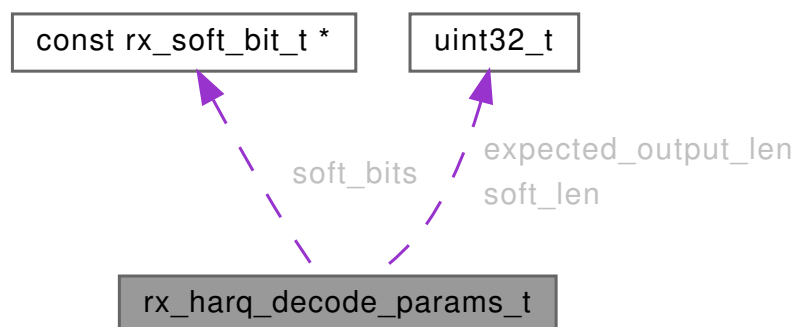
uint8_t	fec_enabled	Non-zero to enable FEC encoding/decoding
uint8_t	max_combines	Maximum combining attempts (0 = default)
uint8_t	max_retries	Maximum transmission attempts (0 = default)

5.1.2.4 struct rx_harq_decode_params_t

HARQ decode parameters.

Definition at line 330 of file [rx_harq.h](#).

Collaboration diagram for rx_harq_decode_params_t:



Data Fields

uint32_t	expected_output_len	Expected decoded length
----------	---------------------	-------------------------

const rx_soft_bit_t *	soft_bits	Received soft bits
uint32_t	soft_len	Number of soft bits

5.1.3 Enumeration Type Documentation

5.1.3.1 rx_harq_params_t

enum [rx_harq_params_t](#) : uint16_t

HARQ protocol parameters.

Enumerator

k_harq_max_payload	Maximum payload size in bytes
k_harq_default_retries	Default maximum retries
k_harq_default_combines	Default max combining attempts
k_harq_soft_buffer_size	Max soft buffer size for Chase Combining $(1024*8+6)*2 = 16396$ bits, rounded to 16400
k_harq_survivors_buffer_size	Survivors buffer size

Definition at line 168 of file [rx_harq.h](#).

```

00168      : uint16_t {
00169  k_harq_max_payload      = 1024, /**< Maximum payload size in bytes */
00170  k_harq_default_retries  = 3,    /**< Default maximum retries */
00171  k_harq_default_combines = 3,    /**< Default max combining attempts */
00172  k_harq_soft_buffer_size = 16400, /**< Max soft buffer size for Chase Combining
00173                                   * (1024*8+6)*2 = 16396 bits, rounded to 16400 */
00174  k_harq_survivors_buffer_size = k_harq_soft_buffer_size / 2U, /**< Survivors buffer size */
00175 } rx_harq_params_t;

```

5.1.3.2 rx_harq_state_t

enum [rx_harq_state_t](#) : uint8_t

HARQ state machine states.

Enumerator

k_harq_state_idle	Ready for new transmission
k_harq_state_waiting_ack	Waiting for ACK
k_harq_state_combining	Combining retransmissions
k_harq_state_error	Unrecoverable error

Definition at line 185 of file [rx_harq.h](#).

```

00185      : uint8_t {
00186  k_harq_state_idle      = 0, /**< Ready for new transmission */
00187  k_harq_state_waiting_ack = 1, /**< Waiting for ACK */
00188  k_harq_state_combining  = 2, /**< Combining retransmissions */
00189  k_harq_state_error      = 3, /**< Unrecoverable error */
00190 } rx_harq_state_t;

```

5.1.4 Function Documentation

5.1.4.1 rx_chase_combiner_add()

```
rx_err_t rx_chase_combiner_add (
    rx_chase_combiner_t * combiner,
    const rx_soft_bit_t * soft_bits,
    uint32_t len)
```

Add soft bits from a transmission attempt.

Accumulates soft bits element-wise. First call sets expected length.

Parameters

in,out	<i>combiner</i>	Initialized combiner handle
in	<i>soft_bits</i>	Received soft bits
in	<i>len</i>	Number of soft bits

Returns

k_rx_ok on success
k_rx_err_invalid_arg if any pointer is nullptr or len is 0
k_rx_err_invalid_size if len doesn't match expected length
k_rx_err_busy if max combines reached

Definition at line 188 of file rx_harq.c.

```
00189 {
00190     if (combiner == nullptr || soft_bits == nullptr) {
00191         return k_rx_err_invalid_arg;
00192     }
00193
00194     if (combiner->initialized == k_harq_false) {
00195         return k_rx_err_invalid_state;
00196     }
00197
00198     if (len == 0) {
00199         return k_rx_err_invalid_arg;
00200     }
00201
00202     if (len > k_harq_soft_buffer_size) {
00203         return k_rx_err_invalid_size;
00204     }
00205
00206     if (combiner->count >= combiner->max_combines) {
00207         return k_rx_err_busy;
00208     }
00209
00210     /* First transmission sets the expected length */
00211     if (combiner->count == 0) {
00212         combiner->expected_len = len;
00213         /* Clear accumulator for new frame */
00214         memset(combiner->accumulated, 0, len * sizeof(int16_t));
00215     } else if (len != combiner->expected_len) {
00216         return k_rx_err_invalid_size;
00217     }
00218
00219     /*
00220     * Element-wise addition (accumulate).
00221     *
00222     * Loop bound verification (NASA Rule 2):
00223     * - len is validated: len > 0 (line 62-64) AND len <= k_harq_soft_buffer_size (line 66-68)
00224     * - Maximum iterations: k_harq_soft_buffer_size (16400)
00225     * - Array bounds: accumulated[i] and soft_bits[i] are safe for i < len
00226     */
00227     for (uint32_t i = 0; i < len; i++) {
```

```

00228     combiner->accumulated[i] += (int16_t)soft_bits[i];
00229 }
00230
00231 combiner->count++;
00232 return k_rx_ok;
00233 }

```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), [k_harq_soft_buffer_size](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.1.4.2 rx_chase_combiner_can_add()

```

bool rx_chase_combiner_can_add (
    const rx\_chase\_combiner\_t * combiner) [nodiscard]

```

Check if more transmissions can be combined.

Parameters

in	<i>combiner</i>	Pointer to combiner handle
----	-----------------	----------------------------

Returns

true if count < max_combines

Check if more transmissions can be combined.

Parameters

in	<i>combiner</i>	Pointer to Chase combiner instance
----	-----------------	------------------------------------

Returns

true if more transmissions can be added, false otherwise

Definition at line 301 of file [rx_harq.c](#).

```

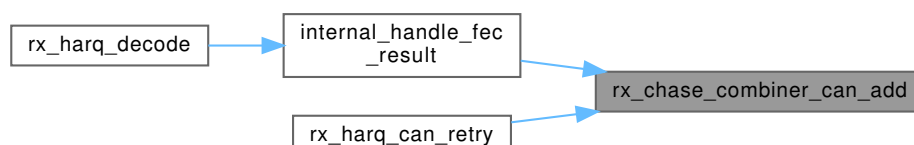
00302 {
00303     if (combiner == nullptr || combiner->initialized == k_harq_false) {
00304         return false;
00305     }
00306     return combiner->count < combiner->max_combines;
00307 }

```

References [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [internal_handle_fec_result\(\)](#), and [rx_harq_can_retry\(\)](#).

Here is the caller graph for this function:



5.1.4.3 rx_chase_combiner_combined()

```
rx_err_t rx_chase_combiner_combined (
    const rx_chase_combiner_t * combiner,
    rx_soft_bit_t * output,
    uint32_t * len) [nodiscard]
```

Get combined soft bits for decoding.

Returns accumulated values clamped to SoftBit range [-127, +127].

Parameters

in	<i>combiner</i>	Initialized combiner handle
out	<i>output</i>	Output buffer for combined soft bits
out	<i>len</i>	Number of combined soft bits

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if any pointer is nullptr
`k_rx_err_invalid_state` if no soft bits have been added

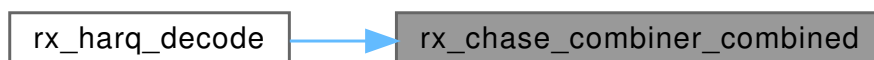
Definition at line 235 of file `rx_harq.c`.

```
00238 {
00239     if (combiner == nullptr || output == nullptr || len == nullptr) {
00240         return k_rx_err_invalid_arg;
00241     }
00242     if (combiner->initialized == k_harq_false) {
00243         return k_rx_err_invalid_state;
00244     }
00245     if (combiner->count == 0 || combiner->expected_len == 0) {
00246         return k_rx_err_invalid_state;
00247     }
00248     /*
00249     * Clamp accumulated values to SoftBit range [-127, +127].
00250     *
00251     * Loop bound verification (NASA Rule 2):
00252     * - expected_len is set only via rx_chase_combiner_add() with validated len
00253     * - expected_len > 0 (validated at line 103-105)
00254     * - expected_len <= k_harq_soft_buffer_size (validated when set in add)
00255     * - Maximum iterations: k_harq_soft_buffer_size (16400)
00256     * - Array bounds: accumulated[i] and output[i] are safe for i < expected_len
00257     */
00258     for (uint32_t i = 0; i < combiner->expected_len; i++) {
00259         const int16_t acc = combiner->accumulated[i];
00260         if (acc > k_soft_bit_max) {
00261             output[i] = k_soft_bit_max;
00262         } else if (acc < k_soft_bit_min) {
00263             output[i] = k_soft_bit_min;
00264         } else {
00265             output[i] = (rx_soft_bit_t)acc;
00266         }
00267     }
00268     *len = combiner->expected_len;
00269     return k_rx_ok;
00270 }
```

References `rx_chase_combiner_t::accumulated`, `rx_chase_combiner_t::count`, `rx_chase_combiner_t::expected_len`, `rx_chase_combiner_t::initialized`, and `k_harq_false`.

Referenced by `rx_harq_decode()`.

Here is the caller graph for this function:



5.1.4.4 rx_chase_combiner_count()

```
uint8_t rx_chase_combiner_count (
    const rx_chase_combiner_t * combiner)
```

Get number of combined transmissions.

Parameters

in	<i>combiner</i>	Pointer to combiner handle
----	-----------------	----------------------------

Returns

Number of transmissions combined

Get number of combined transmissions.

Parameters

in	<i>combiner</i>	Pointer to Chase combiner instance
----	-----------------	------------------------------------

Returns

Number of combined transmissions (0 if combiner is nullptr)

Definition at line 316 of file [rx_harq.c](#).

```
00317 {
00318     if (combiner == nullptr) {
00319         return 0;
00320     }
00321     return combiner->count;
00322 }
```

References [rx_chase_combiner_t::count](#).

5.1.4.5 rx_chase_combiner_deinit()

```
rx_err_t rx_chase_combiner_deinit (
    rx_chase_combiner_t * combiner) [nodiscard]
```

Deinitialize Chase Combiner.

Parameters

in,out	<i>combiner</i>	Pointer to combiner handle
--------	-----------------	----------------------------

Returns

k_rx_ok on success

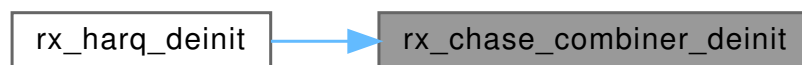
Definition at line 177 of file rx_harq.c.

```
00178 {
00179     if (combiner == nullptr) {
00180         return k_rx_err_invalid_arg;
00181     }
00182     combiner->initialized = k_harq_false;
00184     return k_rx_ok;
00185 }
```

References [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [rx_harq_deinit\(\)](#).

Here is the caller graph for this function:



5.1.4.6 rx_chase_combiner_init()

```
rx_err_t rx_chase_combiner_init (
    rx_chase_combiner_t * combiner,
    uint8_t max_combines) [nodiscard]
```

Initialize Chase Combiner.

Parameters

out	<i>combiner</i>	Pointer to combiner handle
in	<i>max_combines</i>	Maximum number of combining attempts (0 = default)

Returns

k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr

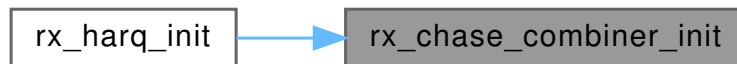
Definition at line 156 of file rx_harq.c.

```
00157 {
00158     if (combiner == nullptr) {
00159         return k_rx_err_invalid_arg;
00160     }
00161     /* Clear accumulator */
00163     memset(combiner->accumulated, 0, sizeof(combiner->accumulated));
00164     combiner->expected_len = 0;
00166     combiner->count = 0;
00167     if (max_combines > k_harq_zero_combines) {
00168         combiner->max_combines = max_combines;
00169     } else {
00170         combiner->max_combines = k_harq_default_combines;
00171     }
00172     combiner->initialized = k_harq_true;
00173     return k_rx_ok;
00175 }
```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_default_combines](#), [k_harq_true](#), [k_harq_zero_combines](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_init\(\)](#).

Here is the caller graph for this function:



5.1.4.7 rx_chase_combiner_reset()

```

rx_err_t rx_chase_combiner_reset (
    rx_chase_combiner_t * combiner) [nodiscard]
  
```

Reset combiner for new frame.

Clears accumulated buffer and count.

Parameters

in,out	<i>combiner</i>	Pointer to combiner handle
--------	-----------------	----------------------------

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if combiner is nullptr
 k_rx_err_invalid_state if combiner is not initialized

Definition at line 276 of file [rx_harq.c](#).

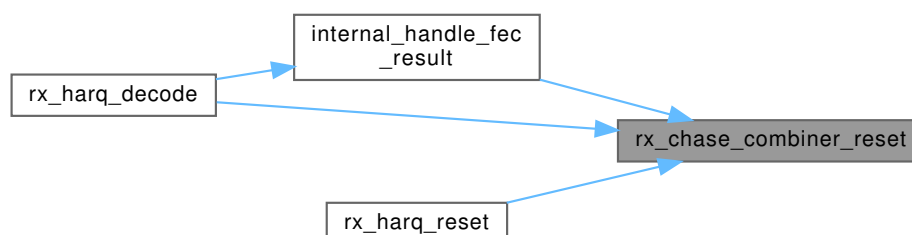
```

00277 {
00278     if (combiner == nullptr) {
00279         return k_rx_err_invalid_arg;
00280     }
00281
00282     if (combiner->initialized == k_harq_false) {
00283         return k_rx_err_invalid_state;
00284     }
00285
00286     /* Clear the entire accumulator to prevent stale data */
00287     memset(combiner->accumulated, 0, sizeof(combiner->accumulated));
00288     combiner->count = 0;
00289     combiner->expected_len = 0;
00290
00291     return k_rx_ok;
00292 }
  
```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [internal_handle_fec_result\(\)](#), [rx_harq_decode\(\)](#), and [rx_harq_reset\(\)](#).

Here is the caller graph for this function:



5.1.4.8 rx_harq_can_retry()

```
bool rx_harq_can_retry (
    const rx_harq_handle_t * harq) [nodiscard]
```

Check if more retries are available.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

true if `retry_count < max_retries`

Check if more retries are available.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

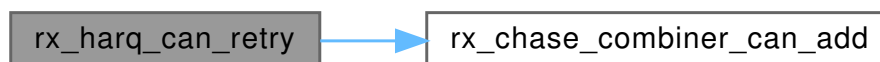
true if retry is possible, false otherwise

Definition at line 695 of file `rx_harq.c`.

```
00696 {
00697     if (harq == nullptr || harq->initialized == k_harq_false) {
00698         return false;
00699     }
00700     return (harq->retry_count < harq->max_retries) && rx_chase_combiner_can_add(&harq->combiner);
00701 }
```

References `rx_harq_handle_t::combiner`, `rx_harq_handle_t::initialized`, `k_harq_false`, `rx_harq_handle_t::max_retries`, `rx_harq_handle_t::retry_count`, and `rx_chase_combiner_can_add()`.

Here is the call graph for this function:



5.1.4.9 rx_harq_decode()

```
rx_err_t rx_harq_decode (
    rx_harq_handle_t * harq,
    const rx_harq_decode_params_t * params,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]
```

Decode soft bits with combining and FEC.

Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.

Parameters

in	<i>harq</i>	HARQ handle
----	-------------	-------------

in	<i>params</i>	Decode parameters
out	<i>output</i>	Decoded output buffer
out	<i>output_len</i>	Actual decoded length

Returns

k_rx_ok on successful decode
 k_rx_err_protocol_error if decode failed (more retries possible)
 k_rx_err_invalid_arg if any pointer is nullptr
 k_rx_err_invalid_state if not initialized

Definition at line 614 of file [rx_harq.c](#).

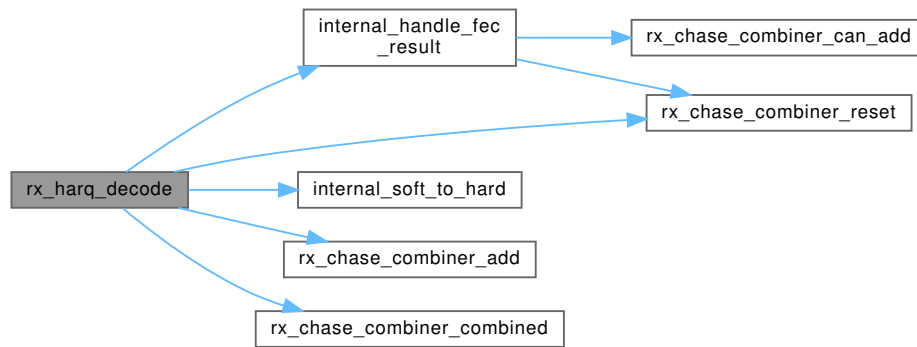
```

00618 {
00619     if (harq == nullptr || params == nullptr || output == nullptr || output_len == nullptr) {
00620         return k_rx_err_invalid_arg;
00621     }
00622     if (harq->initialized == k_harq_false) {
00623         return k_rx_err_invalid_state;
00624     }
00625     if (params->soft_bits == nullptr || params->soft_len == 0) {
00626         return k_rx_err_invalid_arg;
00627     }
00628
00629     /* Add soft bits to combiner */
00630     rx_err_t err = rx_chase_combiner_add(&harq->combiner, params->soft_bits, params->soft_len);
00631     if (err != k_rx_ok && err != k_rx_err_busy) {
00632         return err;
00633     }
00634
00635     /* Get combined soft bits into handle's buffer (thread-safe) */
00636     uint32_t combined_len = 0U;
00637     err = rx_chase_combiner_combined(&harq->combiner, harq->decode_buffer, &combined_len);
00638     if (err != k_rx_ok) {
00639         return err;
00640     }
00641
00642     /* Non-FEC path: direct soft-to-hard conversion */
00643     if (harq->fec_enabled == k_harq_false) {
00644         err = internal_soft_to_hard(harq->decode_buffer,
00645                                     combined_len,
00646                                     params->expected_output_len,
00647                                     output,
00648                                     output_len);
00649         if (err != k_rx_ok) {
00650             return err;
00651         }
00652         harq->retry_count++;
00653         /* Ignore return - combiner is already validated via harq->initialized */
00654         (void)rx_chase_combiner_reset(&harq->combiner);
00655         harq->state = k_harq_state_idle;
00656         return k_rx_ok;
00657     }
00658
00659     /* FEC path: decode combined soft bits */
00660     const rx_fec_decode_soft_params_t decode_params = {
00661         .soft_bits      = harq->decode_buffer,
00662         .soft_len       = combined_len,
00663         .expected_output_len = params->expected_output_len,
00664         .output         = output,
00665         .output_len     = output_len,
00666     };
00667
00668     err = rx_fec_decode_soft(&harq->decoder, &decode_params);
00669
00670     return internal_handle_fec_result(harq, err);
00671 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decode_buffer](#), [rx_harq_handle_t::decoder](#), [rx_harq_decode_params_t::](#)
[rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [internal_handle_fec_result\(\)](#), [internal_soft_to_hard\(\)](#),
[k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_add\(\)](#), [rx_chase_combiner_combined\(\)](#),
[rx_chase_combiner_reset\(\)](#), [rx_harq_decode_params_t::soft_bits](#), [rx_harq_decode_params_t::soft_len](#), and
[rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.1.4.10 rx_harq_deinit()

```
rx_err_t rx_harq_deinit (
    rx_harq_handle_t * harq) [nodiscard]
```

Deinitialize HARQ handle.

Parameters

in,out	<i>harq</i>	Pointer to HARQ handle
--------	-------------	------------------------

Returns

k_rx_ok on success

Deinitialize HARQ handle.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

k_rx_ok on success, k_rx_err_invalid_arg if harq is nullptr

Definition at line 389 of file rx_harq.c.

```

00390 {
00391     if (harq == nullptr) {
00392         return k_rx_err_invalid_arg;
00393     }
00394
00395     rx_err_t err = k_rx_ok;
00396
00397     if (harq->fec_enabled == k_harq_true) {
00398         rx_err_t fec_enc_err = rx_fec_encoder_deinit(&harq->encoder);
00399         if (fec_enc_err != k_rx_ok) {
00400             err = fec_enc_err; /* Save first error */
00401         }
00402         rx_err_t fec_dec_err = rx_fec_decoder_deinit(&harq->decoder);
00403         if (fec_dec_err != k_rx_ok && err == k_rx_ok) {
00404             err = fec_dec_err; /* Save first error */
00405         }
00406     }
00407
00408     rx_err_t combiner_err = rx_chase_combiner_deinit(&harq->combiner);
```

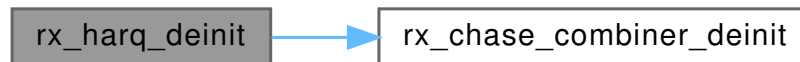
```

00409 if (combiner_err != k_rx_ok && err == k_rx_ok) {
00410     err = combiner_err; /* Save first error */
00411 }
00412
00413 harq->initialized = k_harq_false;
00414 return err;
00415 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_true](#), and [rx_chase_combiner_deinit\(\)](#).

Here is the call graph for this function:



5.1.4.11 rx_harq_encode()

```

rx_err_t rx_harq_encode (
    const rx_harq_handle_t * harq,
    const uint8_t * payload,
    uint32_t payload_len,
    uint8_t * output,
    uint32_t output_size,
    uint32_t * output_len) [nodiscard]

```

Encode payload with FEC (if enabled).

Parameters

in	<i>harq</i>	HARQ handle
in	<i>payload</i>	Input payload
in	<i>payload_len</i>	Payload length
out	<i>output</i>	Output buffer (must be large enough for encoded data)
in	<i>output_size</i>	Size of output buffer in bytes
out	<i>output_len</i>	Actual output length

Returns

- k_rx_ok on success
- k_rx_err_invalid_arg if any pointer is nullptr
- k_rx_err_invalid_size if output buffer too small
- k_rx_err_invalid_state if not initialized

Definition at line 462 of file [rx_harq.c](#).

```

00468 {
00469     if (harq == nullptr || payload == nullptr || output == nullptr || output_len == nullptr) {
00470         return k_rx_err_invalid_arg;
00471     }
00472
00473     if (harq->initialized == k_harq_false) {
00474         return k_rx_err_invalid_state;
00475     }
00476

```

```

00477 if (payload_len == 0 || payload_len > k_harq_max_payload) {
00478     return k_rx_err_invalid_size;
00479 }
00480
00481 /* If FEC is disabled, just copy payload */
00482 if (harq->fec_enabled == k_harq_false) {
00483     /* Validate output buffer can hold payload */
00484     if (output_size < payload_len) {
00485         return k_rx_err_invalid_size;
00486     }
00487     memcpy(output, payload, payload_len);
00488     *output_len = payload_len;
00489     return k_rx_ok;
00490 }
00491
00492 /*
00493  * FEC encode: Rate 1/2 convolutional code doubles output size.
00494  * Required buffer: (payload_len * 8 + 6) * 2 bits = (payload_len * 8 + 6) / 4 bytes
00495  * Simplified worst case: payload_len * 2 + 2 bytes
00496  */
00497 const uint32_t min_output_size = (payload_len * k_harq_fec_rate_multiplier) + k_harq_tail_bytes;
00498 if (output_size < min_output_size) {
00499     return k_rx_err_invalid_size;
00500 }
00501
00502 /* FEC encode */
00503 return rx_fec_encode(&harq->encoder, payload, payload_len, output, output_len);
00504 }

```

References [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_fec_rate_multiplier](#), [k_harq_max_payload](#), and [k_harq_tail_bytes](#).

5.1.4.12 rx_harq_get_retry_count()

```

uint8_t rx_harq_get_retry_count (
    const rx_harq_handle_t * harq)

```

Get current retry count.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

Current retry count

Get current retry count.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

Number of retries (0 if harq is nullptr)

Definition at line 680 of file [rx_harq.c](#).

```

00681 {
00682     if (harq == nullptr) {
00683         return 0;
00684     }
00685     return harq->retry_count;
00686 }

```

References [rx_harq_handle_t::retry_count](#).

5.1.4.13 rx_harq_get_state()

```
rx_harq_state_t rx_harq_get_state (
    const rx_harq_handle_t * harq)
```

Get current HARQ state.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

Current HARQ state

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

Current state, or k_harq_state_error if harq is nullptr

Definition at line 424 of file rx_harq.c.

```
00425 {
00426     if (harq == nullptr) {
00427         return k_harq_state_error;
00428     }
00429     return harq->state;
00430 }
```

References [k_harq_state_error](#), and [rx_harq_handle_t::state](#).

5.1.4.14 rx_harq_init()

```
rx_err_t rx_harq_init (
    rx_harq_handle_t * harq,
    const rx_harq_config_t * config) [nodiscard]
```

Initialize HARQ handle.

Parameters

out	<i>harq</i>	Pointer to HARQ handle
in	<i>config</i>	Configuration (NULL for defaults)

Returns

k_rx_ok on success

k_rx_err_invalid_arg if harq is nullptr

Definition at line 329 of file rx_harq.c.

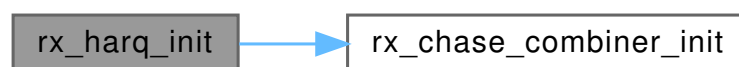
```

00330 {
00331     if (harq == nullptr) {
00332         return k_rx_err_invalid_arg;
00333     }
00334
00335     /* Initialize state */
00336     harq->state = k_harq_state_idle;
00337     harq->tx_sequence = 0;
00338     harq->rx_sequence = 0;
00339     harq->retry_count = 0;
00340
00341     /* Apply configuration */
00342     if (config != nullptr) {
00343         if (config->max_retries > k_harq_zero_retries) {
00344             harq->max_retries = config->max_retries;
00345         } else {
00346             harq->max_retries = k_harq_default_retries;
00347         }
00348         harq->fec_enabled = config->fec_enabled;
00349     } else {
00350         harq->max_retries = k_harq_default_retries;
00351         harq->fec_enabled = k_harq_true; /* FEC enabled by default */
00352     }
00353
00354     /* Initialize Chase Combiner */
00355     const bool has_combines = (config != nullptr && config->max_combines > k_harq_zero_combines);
00356     const uint8_t max_combines = has_combines ? config->max_combines : k_harq_default_combines;
00357     rx_err_t err = rx_chase_combiner_init(&harq->combiner, max_combines);
00358     if (err != k_rx_ok) {
00359         return err;
00360     }
00361
00362     /* Initialize FEC encoder if enabled */
00363     if (harq->fec_enabled == k_harq_true) {
00364         err = rx_fec_encoder_init(&harq->encoder);
00365         if (err != k_rx_ok) {
00366             return err;
00367         }
00368
00369         /* Initialize FEC decoder with survivors buffer */
00370         err =
00371             rx_fec_decoder_init(&harq->decoder, harq->decoder_survivors, k_harq_survivors_buffer_size);
00372         if (err != k_rx_ok) {
00373             (void)rx_fec_encoder_deinit(&harq->encoder); /* Cleanup, ignore errors */
00374             return err;
00375         }
00376     }
00377
00378     harq->initialized = k_harq_true;
00379     return k_rx_ok;
00380 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::decoder_survivors](#), [rx_harq_handle_t::encoder](#), [rx_harq_config_t::fec_enabled](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_default_combines](#), [k_harq_default_retries](#), [k_harq_state_idle](#), [k_harq_survivors_buffer_size](#), [k_harq_true](#), [k_harq_zero_combines](#), [k_harq_zero_retries](#), [rx_harq_config_t::max_combines](#), [rx_harq_config_t::max_retries](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_init\(\)](#), [rx_harq_handle_t::rx_sequence](#), [rx_harq_handle_t::state](#), and [rx_harq_handle_t::tx_sequence](#).

Here is the call graph for this function:



5.1.4.15 rx_harq_reset()

```
rx_err_t rx_harq_reset (
    rx_harq_handle_t * harq) [nodiscard]
```

Reset HARQ for new transaction.

Resets state to idle and clears combiner.

Parameters

in,out	<i>harq</i>	Pointer to HARQ handle
--------	-------------	------------------------

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if harq is nullptr
 k_rx_err_invalid_state if harq is not initialized

Reset HARQ for new transaction.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

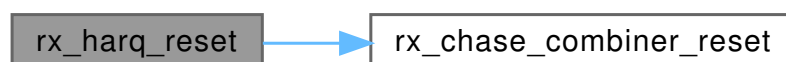
k_rx_ok on success
 k_rx_err_invalid_arg if harq is nullptr
 k_rx_err_invalid_state if not initialized

Definition at line 441 of file [rx_harq.c](#).

```
00442 {
00443     if (harq == nullptr) {
00444         return k_rx_err_invalid_arg;
00445     }
00446
00447     if (harq->initialized == k_harq_false) {
00448         return k_rx_err_invalid_state;
00449     }
00450
00451     harq->state = k_harq_state_idle;
00452     harq->retry_count = 0;
00453
00454     const rx_err_t err = rx_chase_combiner_reset(&harq->combiner);
00455     if (err != k_rx_ok) {
00456         return err;
00457     }
00458
00459     return k_rx_ok;
00460 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_reset\(\)](#), and [rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.2 rx_harq.h

[Go to the documentation of this file.](#)

```

00001 /* lib/rx_harq/inc/rx_harq.h */
00002
00003 /**
00004  * @file rx_harq.h
00005  * @brief Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N
00006  *
00007  * @details
00008  * Implements **Chase Combining HARQ Type I** for reliable communication over
00009  * lossy channels. Combines FEC (Forward Error Correction) with ARQ (Automatic
00010  * Repeat Request) to achieve both error correction and detection.
00011  *
00012  * ## HARQ Protocol Overview
00013  *
00014  * HARQ merges FEC and ARQ:
00015  * - **FEC**: Corrects random bit errors via convolutional coding
00016  * - **ARQ**: Requests retransmission when FEC fails
00017  * - **Chase Combining**: Accumulates soft bits from multiple transmissions
00018  *
00019  * ## Chase Combining Theory
00020  *
00021  * When a transmission fails to decode, instead of discarding it, the receiver:
00022  * 1. **Stores** soft bits from failed attempt
00023  * 2. **Requests** retransmission (NACK)
00024  * 3. **Adds** new soft bits to accumulated values (element-wise)
00025  * 4. **Decodes** combined soft bits (higher SNR -> better chance)
00026  *
00027  * ### Mathematical Model
00028  *
00029  * For @f$ n $ transmissions of the same data, the combined soft bit is:
00030  * @f[
00031  * s_{\text{combined}}[i] = \sum_{k=1}^n s_k[i]
00032  * @f]
00033  *
00034  * This provides **diversity gain**: independent noise samples are averaged out.
00035  *
00036  * ### SNR Improvement
00037  *
00038  * Assuming independent noise across transmissions, SNR improves as:
00039  * @f[
00040  * \text{SNR}_{\text{combined}} \approx n \cdot \text{SNR}_{\text{single}}
00041  * @f]
00042  *
00043  * In dB: @f$ 10 \log_{10}(n) $ gain (e.g., 3 dB for 2 transmissions, 4.8 dB for 3).
00044  *
00045  * ## Protocol Sequence
00046  *
00047  * @msc
00048  * Transmitter, Channel, Receiver;
00049  *
00050  * --- [label="Transmission 1"];
00051  * Transmitter box Transmitter [label="Encode with FEC\n100 bytes -> 202 bytes"];
00052  * Transmitter => Channel [label="Soft bits (1st attempt)"];
00053  * Channel box Channel [label="Add noise\nBER = 10^-3"];
00054  * Channel => Receiver [label="Noisy soft bits"];
00055  * Receiver box Receiver [label="Decode attempt\nFAILED"];
00056  * Receiver box Receiver [label="Store soft bits\nin combiner"];
00057  * Receiver => Transmitter [label="NACK (request retry)"];
00058  *
00059  * --- [label="Transmission 2"];
00060  * Transmitter => Channel [label="Soft bits (2nd attempt)"];
00061  * Channel box Channel [label="Independent noise\nBER = 10^-3"];
00062  * Channel => Receiver [label="Noisy soft bits"];
00063  * Receiver box Receiver [label="Add to combiner\nCombine attempts 1+2"];
00064  * Receiver box Receiver [label="Decode combined\nSUCCESS (3 dB gain)"];
00065  * Receiver => Transmitter [label="ACK (success)"];
00066  * Transmitter box Transmitter [label="Move to next frame"];
00067  * @endmsc
00068  *
00069  * ## HARQ State Machine
00070  *
00071  * @startuml
00072  * [*] --> Idle
00073  * Idle --> WaitingAck : tx_encode()
00074  * WaitingAck --> Idle : rx_ACK()
00075  * WaitingAck --> Combining : rx_NACK()
00076  * Combining --> Idle : decode_success()
00077  * Combining --> Error : max_retries_reached()
00078  * Error --> Idle : reset()
00079  * @enduml
00080  *
00081  * **State Descriptions:**
00082  * - **Idle**: Ready for new transmission, no pending frames

```

```

00083 * - **WaitingAck**: Waiting for ACK/NACK from receiver
00084 * - **Combining**: Accumulating retransmissions, attempting decode
00085 * - **Error**: Unrecoverable error (exceeded retry limit)
00086 *
00087 * **Transition Guards**:
00088 * - WaitingAck -> Combining: Only if retry_count < max_retries
00089 * - Combining -> Error: When retry_count >= max_retries AND decode still fails
00090 *
00091 * ## Performance Characteristics
00092 *
00093 * | Metric | Value |
00094 * |-----|-----|
00095 * | Throughput (1 tx) | 100 / 202 ~ 49.5% (FEC overhead) |
00096 * | Throughput (2 tx avg) | 100 / 404 ~ 24.8% (2x retransmissions) |
00097 * | Latency (1 tx) | ~1 ms encode + transmit |
00098 * | Latency (2 tx) | ~2-5 ms (includes RTT for NACK) |
00099 * | Memory (per handle) | ~82 KB (dominated by FEC decoder) |
00100 *
00101 * ## Features
00102 *
00103 * - [OK] **Chase Combining**: Soft bit accumulation (int16 to prevent overflow)
00104 * - [OK] **FEC Integration**: Optional K=7 rate 1/2 convolutional code
00105 * - [OK] **Configurable Retries**: 1-255 attempts (default: 3)
00106 * - [OK] **Static Allocation**: Zero malloc (NASA Rule 3)
00107 * - [OK] **Thread Safety**: Safe if handles not shared
00108 *
00109 * @par Hardware Requirements
00110 * | Resource | Usage |
00111 * |-----|-----|
00112 * | CPU | RX72N @ 240 MHz |
00113 * | RAM | ~82 KB per HARQ handle |
00114 * | Flash | ~8 KB (code size with FEC) |
00115 *
00116 * @par Module Dependencies
00117 * - [rx_fec.h](rx_fec.h.html): FEC encoder/decoder
00118 * - [rx_err.h](rx_err.h.html): Error codes
00119 *
00120 * @par NASA Power of 10 Compliance
00121 * - **Rule 1**: [OK] No recursion, goto, setjmp/longjmp
00122 * - **Rule 2**: [OK] All loops bounded (k_harq_soft_buffer_size)
00123 * - **Rule 3**: [OK] Zero dynamic allocation
00124 * - **Rule 4**: [OK] Functions <= 60 lines
00125 * - **Rule 5**: [OK] All functions have >=2 pre/postconditions
00126 *
00127 * @note **Thread Safety**: HARQ handles are NOT thread-safe. Caller must
00128 * provide synchronization if sharing handles between threads.
00129 *
00130 * @warning Soft bit accumulation uses int16 (range +/-32768). With max 3 combines
00131 * and soft bits +/-127, accumulator saturates at +/-381 (no overflow).
00132 *
00133 * @see star-gateway/internal/harq/ Go reference implementation
00134 * @see star-gateway/internal/fec/combiner.go Chase Combiner algorithm
00135 * @see docs/sections/01_nanopb_protocol.tex Protocol specification
00136 * @see "Hybrid ARQ Schemes" by S. Lin & D. Costello
00137 *
00138 * @author STAR Team
00139 * @date 2026-01-27
00140 * @copyright Copyright (c) 2026 STAR Project. Licensed under MIT License.
00141 *
00142 * @since Version 1.0.0
00143 * @version 1.0.0
00144 *
00145 * @test test_rx_harq.c Unit tests for HARQ protocol
00146 */
00147
00148 #pragma once
00149
00150 #include <stdbool.h>
00151 #include <stdint.h>
00152
00153 #include "rx_err.h"
00154 #include "rx_fec.h"
00155
00156 #ifdef __cplusplus
00157 extern "C" {
00158 #endif
00159
00160 /*
=====
00161 * HARQ Parameters
00162 *
=====
00163 */
00164
00165 /**
00166 * @brief HARQ protocol parameters
00167 */

```

```

00168 typedef enum : uint16_t {
00169     k_harq_max_payload      = 1024, /**< Maximum payload size in bytes */
00170     k_harq_default_retries  = 3,    /**< Default maximum retries */
00171     k_harq_default_combines = 3,    /**< Default max combining attempts */
00172     k_harq_soft_buffer_size = 16400, /**< Max soft buffer size for Chase Combining
00173                                     * (1024*8+6)*2 = 16396 bits, rounded to 16400 */
00174     k_harq_survivors_buffer_size = k_harq_soft_buffer_size / 2U, /**< Survivors buffer size */
00175 } rx_harq_params_t;
00176
00177 /*
=====
00178  * HARQ State Machine
00179  *
=====
00180  */
00181 /**
00182  * @brief HARQ state machine states
00183  */
00184
00185 typedef enum : uint8_t {
00186     k_harq_state_idle      = 0, /**< Ready for new transmission */
00187     k_harq_state_waiting_ack = 1, /**< Waiting for ACK */
00188     k_harq_state_combining  = 2, /**< Combining retransmissions */
00189     k_harq_state_error      = 3, /**< Unrecoverable error */
00190 } rx_harq_state_t;
00191
00192 /*
=====
00193  * Chase Combiner
00194  *
=====
00195  */
00196 /**
00197  * @brief Chase Combiner for soft bit accumulation
00198  *
00199  * Stores soft bits from failed transmission attempts and combines them
00200  * element-wise (addition) before decoding. Uses int16 accumulators to
00201  * prevent overflow.
00202  */
00203
00204 typedef struct {
00205     int16_t accumulated[k_harq_soft_buffer_size]; /**< Accumulated soft bits */
00206     uint32_t expected_len;                        /**< Expected soft bits len */
00207     uint8_t count;                                /**< Transmissions combined */
00208     uint8_t max_combines;                         /**< Max combining attempts */
00209     uint8_t initialized;                          /**< Non-zero if initialized */
00210 } rx_chase_combiner_t;
00211
00212 /**
00213  * @brief Initialize Chase Combiner
00214  *
00215  * @param[out] combiner Pointer to combiner handle
00216  * @param[in] max_combines Maximum number of combining attempts (0 = default)
00217  * @return k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr
00218  */
00219 [[nodiscard]] rx_err_t rx_chase_combiner_init(rx_chase_combiner_t* combiner, uint8_t max_combines);
00220
00221 /**
00222  * @brief Deinitialize Chase Combiner
00223  *
00224  * @param[in,out] combiner Pointer to combiner handle
00225  * @return k_rx_ok on success
00226  */
00227 [[nodiscard]] rx_err_t rx_chase_combiner_deinit(rx_chase_combiner_t* combiner);
00228
00229 /**
00230  * @brief Add soft bits from a transmission attempt
00231  *
00232  * Accumulates soft bits element-wise. First call sets expected length.
00233  *
00234  * @param[in,out] combiner Initialized combiner handle
00235  * @param[in] soft_bits Received soft bits
00236  * @param[in] len Number of soft bits
00237  *
00238  * @return k_rx_ok on success
00239  * @return k_rx_err_invalid_arg if any pointer is nullptr or len is 0
00240  * @return k_rx_err_invalid_size if len doesn't match expected length
00241  * @return k_rx_err_busy if max combines reached
00242  */
00243 rx_err_t
00244 rx_chase_combiner_add(rx_chase_combiner_t* combiner, const rx_soft_bit_t* soft_bits, uint32_t len);
00245
00246 /**
00247  * @brief Get combined soft bits for decoding
00248  *
00249  * Returns accumulated values clamped to SoftBit range [-127, +127].
00250  */

```

```

00251 * @param[in] combiner Initialized combiner handle
00252 * @param[out] output Output buffer for combined soft bits
00253 * @param[out] len Number of combined soft bits
00254 *
00255 * @return k_rx_ok on success
00256 * @return k_rx_err_invalid_arg if any pointer is nullptr
00257 * @return k_rx_err_invalid_state if no soft bits have been added
00258 */
00259 [[nodiscard]] rx_err_t rx_chase_combiner_combined(const rx_chase_combiner_t* combiner,
00260                                                    rx_soft_bit_t* output,
00261                                                    uint32_t* len);
00262 /**
00263 * @brief Reset combiner for new frame
00264 *
00265 * Clears accumulated buffer and count.
00266 *
00267 * @param[in,out] combiner Pointer to combiner handle
00268 *
00269 * @return k_rx_ok on success
00270 * @return k_rx_err_invalid_arg if combiner is nullptr
00271 * @return k_rx_err_invalid_state if combiner is not initialized
00272 */
00273 [[nodiscard]] rx_err_t rx_chase_combiner_reset(rx_chase_combiner_t* combiner);
00274 /**
00275 * @brief Check if more transmissions can be combined
00276 *
00277 * @param[in] combiner Pointer to combiner handle
00278 * @return true if count < max_combines
00279 */
00280 [[nodiscard]] bool rx_chase_combiner_can_add(const rx_chase_combiner_t* combiner);
00281 /**
00282 * @brief Get number of combined transmissions
00283 *
00284 * @param[in] combiner Pointer to combiner handle
00285 * @return Number of transmissions combined
00286 */
00287 uint8_t rx_chase_combiner_count(const rx_chase_combiner_t* combiner);
00288 /**
00289 =====
00290 * HARQ Handle
00291 *
00292 =====
00293 */
00294 /**
00295 * @brief HARQ protocol handle
00296 *
00297 * Contains all state for HARQ operation including FEC encoder/decoder
00298 * and Chase Combiner.
00299 */
00300 typedef struct {
00301     rx_harq_state_t state; /**< Current HARQ state */
00302     uint16_t tx_sequence; /**< TX sequence number */
00303     uint16_t rx_sequence; /**< RX sequence number */
00304     uint8_t retry_count; /**< Current retry count */
00305     uint8_t max_retries; /**< Maximum retries allowed */
00306     rx_chase_combiner_t combiner; /**< Chase Combiner */
00307     rx_fec_encoder_t encoder; /**< FEC encoder */
00308     rx_fec_decoder_t decoder; /**< FEC decoder */
00309     uint64_t decoder_survivors[k_harq_survivors_buffer_size]; /**< Decoder buf */
00310     rx_soft_bit_t decode_buffer[k_harq_soft_buffer_size]; /**< Combined soft bits buf */
00311     uint8_t fec_enabled; /**< Non-zero if FEC is enabled */
00312     uint8_t initialized; /**< Non-zero if initialized */
00313 } rx_harq_handle_t;
00314 /**
00315 * @brief HARQ configuration
00316 */
00317 typedef struct {
00318     uint8_t max_retries; /**< Maximum transmission attempts (0 = default) */
00319     uint8_t fec_enabled; /**< Non-zero to enable FEC encoding/decoding */
00320     uint8_t max_combines; /**< Maximum combining attempts (0 = default) */
00321 } rx_harq_config_t;
00322 /**
00323 * @brief HARQ decode parameters
00324 */
00325 typedef struct {
00326     const rx_soft_bit_t* soft_bits; /**< Received soft bits */
00327     uint32_t soft_len; /**< Number of soft bits */
00328     uint32_t expected_output_len; /**< Expected decoded length */
00329 } rx_harq_decode_params_t;
00330
00331

```

```

00336 /**
00337  * @brief Initialize HARQ handle
00338  *
00339  * @param[out] harq Pointer to HARQ handle
00340  * @param[in]  config Configuration (NULL for defaults)
00341  *
00342  * @return k_rx_ok on success
00343  * @return k_rx_err_invalid_arg if harq is nullptr
00344  */
00345 [[nodiscard]] rx_err_t rx_harq_init(rx_harq_handle_t* harq, const rx_harq_config_t* config);
00346
00347 /**
00348  * @brief Deinitialize HARQ handle
00349  *
00350  * @param[in,out] harq Pointer to HARQ handle
00351  * @return k_rx_ok on success
00352  */
00353 [[nodiscard]] rx_err_t rx_harq_deinit(rx_harq_handle_t* harq);
00354
00355 /**
00356  * @brief Get current HARQ state
00357  *
00358  * @param[in] harq Pointer to HARQ handle
00359  * @return Current HARQ state
00360  */
00361 rx_harq_state_t rx_harq_get_state(const rx_harq_handle_t* harq);
00362
00363 /**
00364  * @brief Reset HARQ for new transaction
00365  *
00366  * Resets state to idle and clears combiner.
00367  *
00368  * @param[in,out] harq Pointer to HARQ handle
00369  *
00370  * @return k_rx_ok on success
00371  * @return k_rx_err_invalid_arg if harq is nullptr
00372  * @return k_rx_err_invalid_state if harq is not initialized
00373  */
00374 [[nodiscard]] rx_err_t rx_harq_reset(rx_harq_handle_t* harq);
00375
00376 /**
00377  * @brief Encode payload with FEC (if enabled)
00378  *
00379  * @param[in]  harq HARQ handle
00380  * @param[in]  payload Input payload
00381  * @param[in]  payload_len Payload length
00382  * @param[out] output Output buffer (must be large enough for encoded data)
00383  * @param[in]  output_size Size of output buffer in bytes
00384  * @param[out] output_len Actual output length
00385  *
00386  * @return k_rx_ok on success
00387  * @return k_rx_err_invalid_arg if any pointer is nullptr
00388  * @return k_rx_err_invalid_size if output buffer too small
00389  * @return k_rx_err_invalid_state if not initialized
00390  */
00391 [[nodiscard]] rx_err_t rx_harq_encode(const rx_harq_handle_t* harq,
00392                                     const uint8_t* payload,
00393                                     uint32_t payload_len,
00394                                     uint8_t* output,
00395                                     uint32_t output_size,
00396                                     uint32_t* output_len);
00397
00398 /**
00399  * @brief Decode soft bits with combining and FEC
00400  *
00401  * Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.
00402  *
00403  * @param[in]  harq HARQ handle
00404  * @param[in]  params Decode parameters
00405  * @param[out] output Decoded output buffer
00406  * @param[out] output_len Actual decoded length
00407  *
00408  * @return k_rx_ok on successful decode
00409  * @return k_rx_err_protocol_error if decode failed (more retries possible)
00410  * @return k_rx_err_invalid_arg if any pointer is nullptr
00411  * @return k_rx_err_invalid_state if not initialized
00412  */
00413 [[nodiscard]] rx_err_t rx_harq_decode(rx_harq_handle_t* harq,
00414                                     const rx_harq_decode_params_t* params,
00415                                     uint8_t* output,
00416                                     uint32_t* output_len);
00417
00418 /**
00419  * @brief Get current retry count
00420  *
00421  * @param[in] harq Pointer to HARQ handle
00422  * @return Current retry count

```

```

00423 */
00424 uint8_t rx_harq_get_retry_count(const rx_harq_handle_t* harq);
00425
00426 /**
00427  * @brief Check if more retries are available
00428  *
00429  * @param[in] harq Pointer to HARQ handle
00430  * @return true if retry_count < max_retries
00431  */
00432 [[nodiscard]] bool rx_harq_can_retry(const rx_harq_handle_t* harq);
00433
00434 #ifdef __cplusplus
00435 }
00436 #endif

```

5.3 libs/rx_harq/src/rx_harq.c File Reference

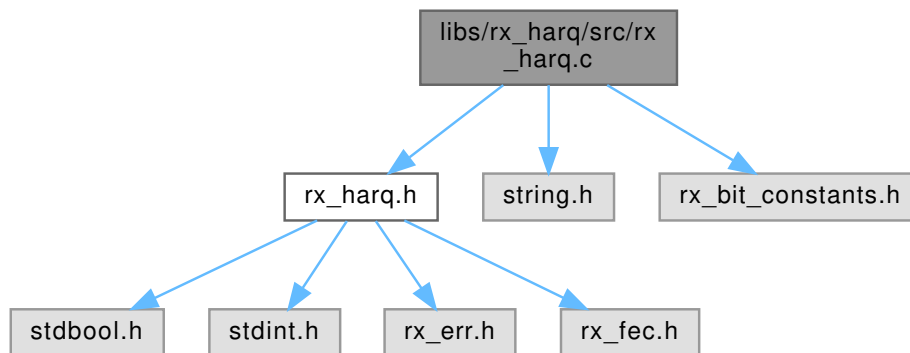
Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

```

#include "rx_harq.h"
#include <string.h>
#include "rx_bit_constants.h"

```

Include dependency graph for rx_harq.c:



Enumerations

- enum [harq_fec_constants_t](#): uint16_t { [k_harq_fec_rate_multiplier](#) = 2 , [k_harq_tail_bytes](#) = 2 }
HARQ FEC size constants.
- enum [harq_cfg_sentinel_t](#): uint8_t { [k_harq_zero_combines](#) = 0 , [k_harq_zero_retries](#) = 0 }
HARQ config validation sentinels – zero values for "not configured, use default".
- enum [harq_bool_t](#): uint8_t { [k_harq_false](#) = 0U , [k_harq_true](#) = 1U }
- enum [bit_constants_t](#): uint8_t { [k_msb_position](#) = [k_rx_bits_per_byte](#) - 1U , [k_rounding_adjustment](#) = [k_rx_bits_per_byte](#) - 1U , [k_soft_bit_zero_thresh](#) = 0 }
Bit position constants for soft-to-hard conversion.

Functions

- rx_err_t [rx_chase_combiner_init](#) (rx_chase_combiner_t *combiner, const uint8_t max_combines)
Initialize Chase Combiner.
- rx_err_t [rx_chase_combiner_deinit](#) (rx_chase_combiner_t *combiner)
Deinitialize Chase Combiner.
- rx_err_t [rx_chase_combiner_add](#) (rx_chase_combiner_t *combiner, const rx_soft_bit_t *soft_bits, uint32_t len)

- Add soft bits from a transmission attempt.*
- rx_err_t [rx_chase_combiner_combined](#) (const [rx_chase_combiner_t](#) *combiner, rx_soft_bit_t *output, uint32_t *len)
- Get combined soft bits for decoding.*
- rx_err_t [rx_chase_combiner_reset](#) ([rx_chase_combiner_t](#) *combiner)
- Reset combiner for new frame.*
- bool [rx_chase_combiner_can_add](#) (const [rx_chase_combiner_t](#) *combiner)
- Check if combiner can accept more transmissions.*
- uint8_t [rx_chase_combiner_count](#) (const [rx_chase_combiner_t](#) *combiner)
- Get number of transmissions combined so far.*
- rx_err_t [rx_harq_init](#) ([rx_harq_handle_t](#) *harq, const [rx_harq_config_t](#) *config)
- Initialize HARQ handle.*
- rx_err_t [rx_harq_deinit](#) ([rx_harq_handle_t](#) *harq)
- Deinitialize HARQ handle and release resources.*
- [rx_harq_state_t](#) [rx_harq_get_state](#) (const [rx_harq_handle_t](#) *harq)
- Get current HARQ state.*
- rx_err_t [rx_harq_reset](#) ([rx_harq_handle_t](#) *harq)
- Reset HARQ state for new transmission.*
- rx_err_t [rx_harq_encode](#) (const [rx_harq_handle_t](#) *harq, const uint8_t *payload, const uint32_t payload_len, uint8_t *output, const uint32_t output_size, uint32_t *output_len)
- Encode payload with FEC (if enabled).*
- static rx_err_t [internal_soft_to_hard](#) (const rx_soft_bit_t *soft_bits, const uint32_t soft_bit_count, const uint32_t max_output_bytes, uint8_t *output, uint32_t *output_len)
- Convert combined soft bits to hard bits (non-FEC path).*
- static rx_err_t [internal_handle_fec_result](#) ([rx_harq_handle_t](#) *harq, const rx_err_t result)
- Handle FEC decode result and update HARQ state.*
- rx_err_t [rx_harq_decode](#) ([rx_harq_handle_t](#) *harq, const [rx_harq_decode_params_t](#) *params, uint8_t *output, uint32_t *output_len)
- Decode soft bits with combining and FEC.*
- uint8_t [rx_harq_get_retry_count](#) (const [rx_harq_handle_t](#) *harq)
- Get number of retries attempted.*
- bool [rx_harq_can_retry](#) (const [rx_harq_handle_t](#) *harq)
- Check if more retries are allowed.*

5.3.1 Detailed Description

Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

Implements Chase Combining HARQ Type I combining FEC with soft bit accumulation across retransmissions. This is a C port of star-gateway/internal/harq/ and fec/combiner.go for bit-exact compatibility between RX72N and RPi5.

5.3.1.1 Implementation Architecture

5.3.1.1.1 Three-Layer Design

1. Chase Combiner ([rx_chase_combiner_t](#))

- Accumulates soft bits element-wise across transmissions
- Uses int16 accumulators (prevents overflow with ≤ 255 combines)
- Clamps output to $[-127, +127]$ range before decoding

2. **FEC Codec** (`rx_fec_encoder_t`, `rx_fec_decoder_t`)

- Optional layer for error correction
- K=7, rate 1/2 convolutional code
- Integrated seamlessly with combiner

3. **HARQ State Machine** (`rx_harq_handle_t`)

- Manages retransmission protocol
- Tracks retry count and sequence numbers
- Coordinates encoder, decoder, and combiner

5.3.1.2 Chase Combining Algorithm

For n transmissions, accumulate soft bits:

$$\text{acc}[i] = \sum_{k=1}^n s_k[i]$$

Then clamp to valid range:

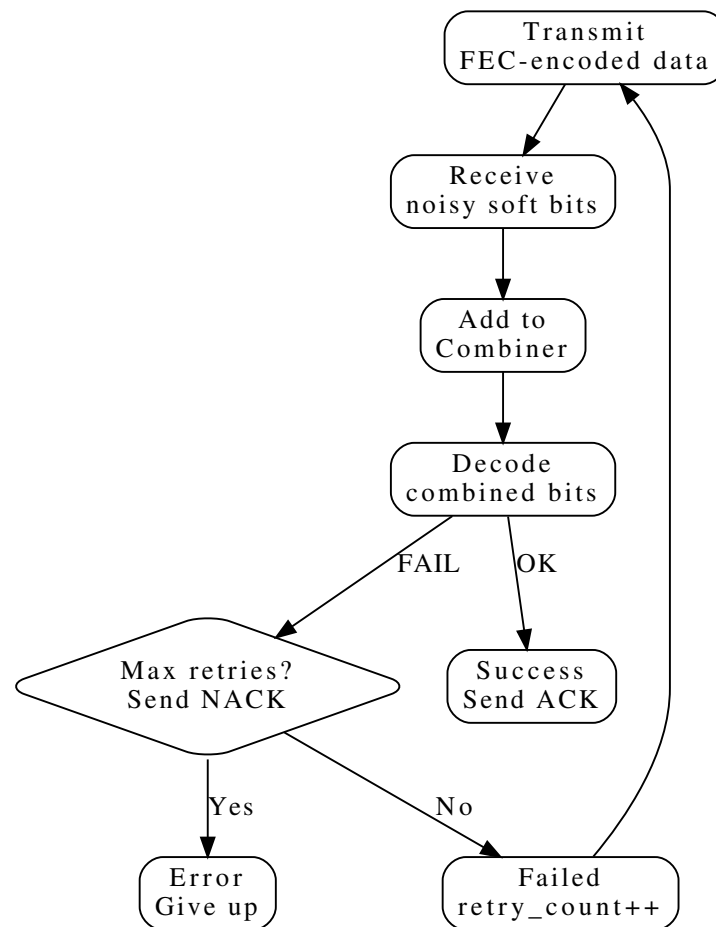
$$s_{\text{combined}}[i] = \begin{cases} +127 & \text{if } \text{acc}[i] > 127 \\ -127 & \text{if } \text{acc}[i] < -127 \\ \text{acc}[i] & \text{otherwise} \end{cases}$$

5.3.1.3 Overflow Prevention

Accumulators use int16 (range +/-32768):

- Max combines: 255
- Max soft bit: +/-127
- Max accumulator value: +/- (255 x 127) = +/-32385
- **Safe:** 32385 < 32768 (no overflow with default max_combines=3)

5.3.1.4 Protocol Flow



Module Dependencies

- `rx_harq.h`: Public API
- `rx_fec.h`: FEC codec
- `rx_bit_constants.h`: Bit manipulation
- `string.h`: `memset`, `memcpy`

NASA Power of 10 Compliance

- **Rule 1:** [OK] No recursion, `goto`, `setjmp/longjmp`
- **Rule 2:** [OK] Loops bounded by `k_harq_soft_buffer_size`
- **Rule 3:** [OK] Zero dynamic allocation
- **Rule 4:** [OK] Functions ≤ 60 lines
- **Rule 5:** [OK] Extensive validation checks

Author

STAR Team

Date

2026-01-27

Copyright

Copyright (c) 2026 STAR Project. Licensed under MIT License.

Since

Version 1.0.0

Version

1.0.0

Definition in file [rx_harq.c](#).

5.3.2 Enumeration Type Documentation

5.3.2.1 bit_constants_t

enum [bit_constants_t](#) : uint8_t

Bit position constants for soft-to-hard conversion.

Enumerator

k_msb_position	Most significant bit position
k_rounding_adjustment	Ceiling division: $(n + 7) / 8$
k_soft_bit_zero_thresh	Threshold for soft-to-hard: ≥ 0 is bit 1

Definition at line 514 of file [rx_harq.c](#).

```
00514      : uint8_t {
00515      k_msb_position      = k_rx_bits_per_byte - 1U, /**< Most significant bit position */
00516      k_rounding_adjustment = k_rx_bits_per_byte - 1U, /**< Ceiling division: (n + 7) / 8 */
00517      k_soft_bit_zero_thresh = 0, /**< Threshold for soft-to-hard:  $\geq 0$  is bit 1 */
00518 } bit_constants_t;
```

5.3.2.2 harq_bool_t

enum [harq_bool_t](#) : uint8_t

Enumerator

k_harq_false	
k_harq_true	

Definition at line 146 of file [rx_harq.c](#).

```
00146      : uint8_t {
00147      k_harq_false = 0U,
00148      k_harq_true  = 1U,
00149 } harq_bool_t;
```

5.3.2.3 harq_cfg_sentinel_t

enum [harq_cfg_sentinel_t](#) : uint8_t

HARQ config validation sentinels – zero values for "not configured, use default".

Used to detect unconfigured (zero) values for max_combines and max_retries fields in HARQ configuration, to fall back to project defaults.

Invariant

k_harq_zero_combines and k_harq_zero_retries equal zero, signifying "use default config" – these sentinels must never be changed to non-zero values.

```
// Checking sentinels when validating config fields:
const rx_harq_config_t config = { .max_combines = 0, .max_retries = 0 };
// max_combines == k_harq_zero_combines -> use k_harq_default_combines
// max_retries == k_harq_zero_retries -> use k_harq_default_retries
uint8_t combines = (config.max_combines > k_harq_zero_combines)
    ? config.max_combines : k_harq_default_combines;
uint8_t retries = (config.max_retries > k_harq_zero_retries)
    ? config.max_retries : k_harq_default_retries;
```

See also

[rx_harq_config_t](#) HARQ configuration structure containing max_combines/max_retries

[rx_harq_init\(\)](#) Applies these sentinels during initialization

Since

Version 1.0.0

Enumerator

k_harq_zero_combines	Sentinel: no max_combines configured -> fall back to default
k_harq_zero_retries	Sentinel: no max_retries configured -> fall back to default

Definition at line 141 of file [rx_harq.c](#).

```
00141     : uint8_t {
00142   k_harq_zero_combines = 0, /**< Sentinel: no max_combines configured -> fall back to default */
00143   k_harq_zero_retries = 0, /**< Sentinel: no max_retries configured -> fall back to default */
00144 } harq_cfg_sentinel_t;
```

5.3.2.4 harq_fec_constants_t

enum [harq_fec_constants_t](#) : uint16_t

HARQ FEC size constants.

Enumerator

k_harq_fec_rate_multiplier	Rate 1/2: encoded length multiplier
k_harq_tail_bytes	Tail overhead in bytes

Definition at line 110 of file [rx_harq.c](#).

```
00110     : uint16_t {
00111   k_harq_fec_rate_multiplier = 2, /**< Rate 1/2: encoded length multiplier */
00112   k_harq_tail_bytes         = 2, /**< Tail overhead in bytes */
00113 } harq_fec_constants_t;
```

5.3.3 Function Documentation

5.3.3.1 internal_handle_fec_result()

```
rx_err_t internal_handle_fec_result (
    rx_harq_handle_t * harq,
    const rx_err_t result) [static]
```

Handle FEC decode result and update HARQ state.

Parameters

in,out	<i>harq</i>	HARQ handle (must not be NULL)
in	<i>result</i>	Decode result from FEC decoder

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if harq is nullptr
 k_rx_err_protocol_error if decode failed but retries available
 result error code if max retries reached

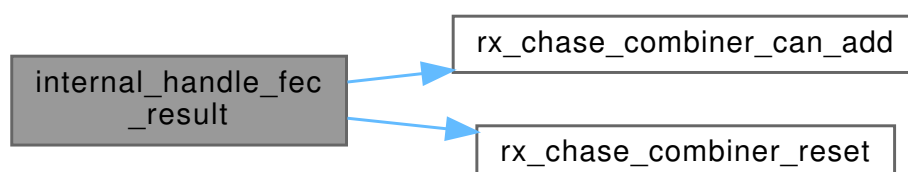
Definition at line 581 of file [rx_harq.c](#).

```
00582 {
00583     /* Critical NULL check */
00584     if (harq == nullptr) {
00585         return k_rx_err_invalid_arg;
00586     }
00587     harq->retry_count++;
00588     if (result == k_rx_ok) {
00589         /* Ignore return - combiner is already validated via harq->initialized */
00590         (void)rx_chase_combiner_reset(&harq->combiner);
00591         harq->state = k_harq_state_idle;
00592         return k_rx_ok;
00593     }
00594     /* Decode failed - check if we can retry */
00595     if (rx_chase_combiner_can_add(&harq->combiner) && harq->retry_count < harq->max_retries) {
00596         harq->state = k_harq_state_combining;
00597         return k_rx_err_protocol_error; /* Need more retransmissions */
00598     }
00599     /* Max retries reached */
00600     (void)rx_chase_combiner_reset(&harq->combiner);
00601     harq->state = k_harq_state_error;
00602     return result;
00603 }
00604
```

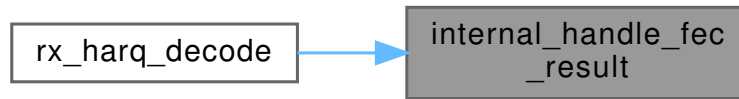
References [rx_harq_handle_t::combiner](#), [k_harq_state_combining](#), [k_harq_state_error](#), [k_harq_state_idle](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_can_add\(\)](#), [rx_chase_combiner_reset\(\)](#), and [rx_harq_handle_t::state](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.3.2 internal_soft_to_hard()

```

rx_err_t internal_soft_to_hard (
    const rx_soft_bit_t * soft_bits,
    const uint32_t soft_bit_count,
    const uint32_t max_output_bytes,
    uint8_t * output,
    uint32_t * output_len) [static]
  
```

Convert combined soft bits to hard bits (non-FEC path).

Parameters

in	<i>soft_bits</i>	Combined soft bits
in	<i>soft_bit_count</i>	Number of soft bits
in	<i>max_output_bytes</i>	Maximum output length (bytes, must be > 0)
out	<i>output</i>	Output hard bits (packed bytes)
out	<i>output_len</i>	Actual output length

Returns

k_rx_ok on success

k_rx_err_invalid_arg if any pointer is nullptr

Definition at line 532 of file [rx_harq.c](#).

```

00537 {
00538     /* Validate all parameters - Critical NULL checks */
00539     if (soft_bits == nullptr || output == nullptr || output_len == nullptr) {
00540         return k_rx_err_invalid_arg;
00541     }
00542     if (max_output_bytes == 0) {
00543         return k_rx_err_invalid_arg;
00544     }
00545     /* Calculate output bytes with ceiling division */
00546     uint32_t out_bytes = (soft_bit_count + k_rounding_adjustment) / k_rx_bits_per_byte;
00547     if (out_bytes > max_output_bytes) {
00548         out_bytes = max_output_bytes;
00549     }
00550     memset(output, 0, out_bytes);
00551     /*
00552     * Loop bound: i < soft_len is validated by prior check that soft_bits != nullptr.
00553     * Loop terminates when i >= soft_len OR when byte index exceeds out_bytes.
00554     * Maximum iterations: min(soft_len, out_bytes * k_rx_bits_per_byte).
00555     */
00556     for (uint32_t i = 0; i < soft_bit_count && (i / k_rx_bits_per_byte) < out_bytes; i++) {
00557         if (soft_bits[i] >= k_soft_bit_zero_thresh) {
00558             const uint32_t byte_idx = i / k_rx_bits_per_byte;
00559             const uint32_t bit_pos = k_msb_position - (i % k_rx_bits_per_byte);
00560             output[byte_idx] |= (uint8_t)(1U << bit_pos);
00561         }
00562     }
00563 }
00564 }
  
```

```

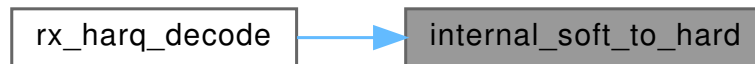
00565
00566  *output_len = out_bytes;
00567  return k_rx_ok;
00568 }

```

References [k_msb_position](#), [k_rounding_adjustment](#), and [k_soft_bit_zero_thresh](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.3.3.3 rx_chase_combiner_add()

```

rx_err_t rx_chase_combiner_add (
    rx_chase_combiner_t * combiner,
    const rx_soft_bit_t * soft_bits,
    uint32_t len)

```

Add soft bits from a transmission attempt.

Accumulates soft bits element-wise. First call sets expected length.

Parameters

in,out	<i>combiner</i>	Initialized combiner handle
in	<i>soft_bits</i>	Received soft bits
in	<i>len</i>	Number of soft bits

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if any pointer is nullptr or len is 0
`k_rx_err_invalid_size` if len doesn't match expected length
`k_rx_err_busy` if max combines reached

Definition at line 188 of file [rx_harq.c](#).

```

00189 {
00190  if (combiner == nullptr || soft_bits == nullptr) {
00191    return k_rx_err_invalid_arg;
00192  }
00193
00194  if (combiner->initialized == k_harq_false) {
00195    return k_rx_err_invalid_state;
00196  }
00197
00198  if (len == 0) {
00199    return k_rx_err_invalid_arg;
00200  }
00201
00202  if (len > k_harq_soft_buffer_size) {
00203    return k_rx_err_invalid_size;
00204  }
00205
00206  if (combiner->count >= combiner->max_combines) {

```

```

00207     return k_rx_err_busy;
00208 }
00209
00210 /* First transmission sets the expected length */
00211 if (combiner->count == 0) {
00212     combiner->expected_len = len;
00213     /* Clear accumulator for new frame */
00214     memset(combiner->accumulated, 0, len * sizeof(int16_t));
00215 } else if (len != combiner->expected_len) {
00216     return k_rx_err_invalid_size;
00217 }
00218
00219 /*
00220  * Element-wise addition (accumulate).
00221  *
00222  * Loop bound verification (NASA Rule 2):
00223  * - len is validated: len > 0 (line 62-64) AND len <= k_harq_soft_buffer_size (line 66-68)
00224  * - Maximum iterations: k_harq_soft_buffer_size (16400)
00225  * - Array bounds: accumulated[i] and soft_bits[i] are safe for i < len
00226  */
00227 for (uint32_t i = 0; i < len; i++) {
00228     combiner->accumulated[i] += (int16_t)soft_bits[i];
00229 }
00230
00231 combiner->count++;
00232 return k_rx_ok;
00233 }

```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), [k_harq_soft_buffer_size](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.3.3.4 rx_chase_combiner_can_add()

```

bool rx_chase_combiner_can_add (
    const rx_chase_combiner_t * combiner) [nodiscard]

```

Check if combiner can accept more transmissions.

Check if more transmissions can be combined.

Parameters

in	<i>combiner</i>	Pointer to Chase combiner instance
----	-----------------	------------------------------------

Returns

true if more transmissions can be added, false otherwise

Definition at line 301 of file [rx_harq.c](#).

```

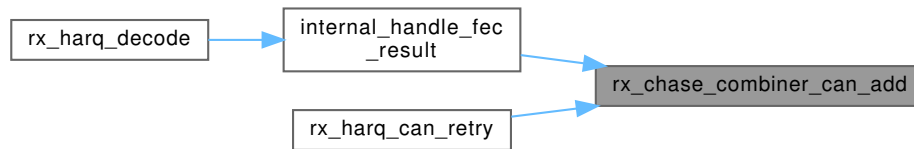
00302 {
00303     if (combiner == nullptr || combiner->initialized == k_harq_false) {
00304         return false;
00305     }
00306     return combiner->count < combiner->max_combines;
00307 }

```

References [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::initialized](#), [k_harq_false](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [internal_handle_fec_result\(\)](#), and [rx_harq_can_retry\(\)](#).

Here is the caller graph for this function:



5.3.3.5 rx_chase_combiner_combined()

```

rx_err_t rx_chase_combiner_combined (
    const rx_chase_combiner_t * combiner,
    rx_soft_bit_t * output,
    uint32_t * len) [nodiscard]

```

Get combined soft bits for decoding.

Returns accumulated values clamped to SoftBit range [-127, +127].

Parameters

in	<i>combiner</i>	Initialized combiner handle
out	<i>output</i>	Output buffer for combined soft bits
out	<i>len</i>	Number of combined soft bits

Returns

- k_rx_ok on success
- k_rx_err_invalid_arg if any pointer is nullptr
- k_rx_err_invalid_state if no soft bits have been added

Definition at line 235 of file rx_harq.c.

```

00238 {
00239     if (combiner == nullptr || output == nullptr || len == nullptr) {
00240         return k_rx_err_invalid_arg;
00241     }
00242
00243     if (combiner->initialized == k_harq_false) {
00244         return k_rx_err_invalid_state;
00245     }
00246
00247     if (combiner->count == 0 || combiner->expected_len == 0) {
00248         return k_rx_err_invalid_state;
00249     }
00250
00251     /*
00252     * Clamp accumulated values to SoftBit range [-127, +127].
00253     *
00254     * Loop bound verification (NASA Rule 2):
00255     * - expected_len is set only via rx_chase_combiner_add() with validated len
00256     * - expected_len > 0 (validated at line 103-105)
00257     * - expected_len <= k_harq_soft_buffer_size (validated when set in add)
00258     * - Maximum iterations: k_harq_soft_buffer_size (16400)
00259     * - Array bounds: accumulated[i] and output[i] are safe for i < expected_len
00260     */
00261     for (uint32_t i = 0; i < combiner->expected_len; i++) {
00262         const int16_t acc = combiner->accumulated[i];
00263         if (acc > k_soft_bit_max) {
00264             output[i] = k_soft_bit_max;
00265         } else if (acc < k_soft_bit_min) {
00266             output[i] = k_soft_bit_min;
00267         } else {
00268             output[i] = (rx_soft_bit_t)acc;
00269         }
00270     }

```

```

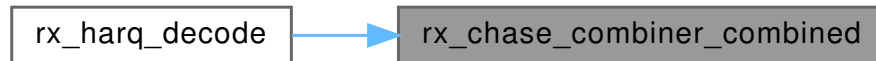
00271
00272  *len = combiner->expected_len;
00273  return k_rx_ok;
00274 }

```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [rx_harq_decode\(\)](#).

Here is the caller graph for this function:



5.3.3.6 rx_chase_combiner_count()

```

uint8_t rx_chase_combiner_count (
    const rx_chase_combiner_t * combiner)

```

Get number of transmissions combined so far.

Get number of combined transmissions.

Parameters

in	<i>combiner</i>	Pointer to Chase combiner instance
----	-----------------	------------------------------------

Returns

Number of combined transmissions (0 if combiner is nullptr)

Definition at line 316 of file [rx_harq.c](#).

```

00317 {
00318  if (combiner == nullptr) {
00319      return 0;
00320  }
00321  return combiner->count;
00322 }

```

References [rx_chase_combiner_t::count](#).

5.3.3.7 rx_chase_combiner_deinit()

```

rx_err_t rx_chase_combiner_deinit (
    rx_chase_combiner_t * combiner) [nodiscard]

```

Deinitialize Chase Combiner.

Parameters

in,out	<i>combiner</i>	Pointer to combiner handle
--------	-----------------	----------------------------

Returns

k_rx_ok on success

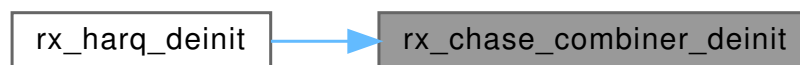
Definition at line 177 of file rx_harq.c.

```
00178 {
00179     if (combiner == nullptr) {
00180         return k_rx_err_invalid_arg;
00181     }
00182     combiner->initialized = k_harq_false;
00184     return k_rx_ok;
00185 }
```

References [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [rx_harq_deinit\(\)](#).

Here is the caller graph for this function:



5.3.3.8 rx_chase_combiner_init()

```
rx_err_t rx_chase_combiner_init (
    rx_chase_combiner_t * combiner,
    uint8_t max_combines) [nodiscard]
```

Initialize Chase Combiner.

Parameters

out	<i>combiner</i>	Pointer to combiner handle
in	<i>max_combines</i>	Maximum number of combining attempts (0 = default)

Returns

k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr

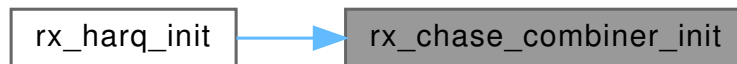
Definition at line 156 of file rx_harq.c.

```
00157 {
00158     if (combiner == nullptr) {
00159         return k_rx_err_invalid_arg;
00160     }
00161     /* Clear accumulator */
00162     memset(combiner->accumulated, 0, sizeof(combiner->accumulated));
00164     combiner->expected_len = 0;
00166     combiner->count = 0;
00167     if (max_combines > k_harq_zero_combines) {
00168         combiner->max_combines = max_combines;
00169     } else {
00170         combiner->max_combines = k_harq_default_combines;
00171     }
00172     combiner->initialized = k_harq_true;
00173     return k_rx_ok;
00174 }
00175 }
```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), [k_harq_default_combines](#), [k_harq_true](#), [k_harq_zero_combines](#), and [rx_chase_combiner_t::max_combines](#).

Referenced by [rx_harq_init\(\)](#).

Here is the caller graph for this function:



5.3.3.9 rx_chase_combiner_reset()

```

rx_err_t rx_chase_combiner_reset (
    rx_chase_combiner_t * combiner) [nodiscard]
  
```

Reset combiner for new frame.

Clears accumulated buffer and count.

Parameters

in,out	<i>combiner</i>	Pointer to combiner handle
--------	-----------------	----------------------------

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if combiner is nullptr
 k_rx_err_invalid_state if combiner is not initialized

Definition at line 276 of file [rx_harq.c](#).

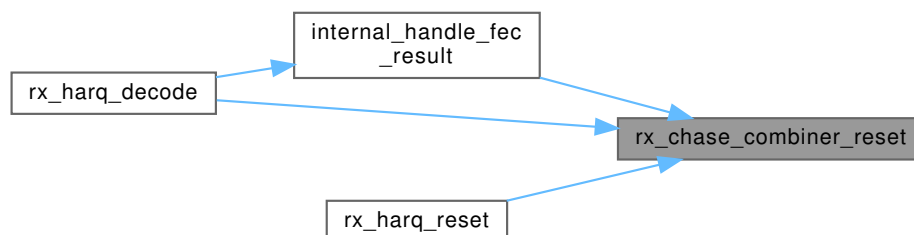
```

00277 {
00278     if (combiner == nullptr) {
00279         return k_rx_err_invalid_arg;
00280     }
00281
00282     if (combiner->initialized == k_harq_false) {
00283         return k_rx_err_invalid_state;
00284     }
00285
00286     /* Clear the entire accumulator to prevent stale data */
00287     memset(combiner->accumulated, 0, sizeof(combiner->accumulated));
00288     combiner->count = 0;
00289     combiner->expected_len = 0;
00290
00291     return k_rx_ok;
00292 }
  
```

References [rx_chase_combiner_t::accumulated](#), [rx_chase_combiner_t::count](#), [rx_chase_combiner_t::expected_len](#), [rx_chase_combiner_t::initialized](#), and [k_harq_false](#).

Referenced by [internal_handle_fec_result\(\)](#), [rx_harq_decode\(\)](#), and [rx_harq_reset\(\)](#).

Here is the caller graph for this function:



5.3.3.10 rx_harq_can_retry()

```
bool rx_harq_can_retry (
    const rx_harq_handle_t * harq) [nodiscard]
```

Check if more retries are allowed.

Check if more retries are available.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

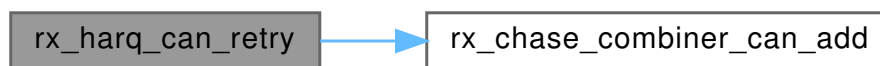
true if retry is possible, false otherwise

Definition at line 695 of file [rx_harq.c](#).

```
00696 {
00697     if (harq == nullptr || harq->initialized == k_harq_false) {
00698         return false;
00699     }
00700     return (harq->retry_count < harq->max_retries) && rx_chase_combiner_can_add(&harq->combiner);
00701 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), and [rx_chase_combiner_can_add\(\)](#).

Here is the call graph for this function:



5.3.3.11 rx_harq_decode()

```
rx_err_t rx_harq_decode (
    rx_harq_handle_t * harq,
    const rx_harq_decode_params_t * params,
    uint8_t * output,
    uint32_t * output_len) [nodiscard]
```

Decode soft bits with combining and FEC.

Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.

Parameters

in	<i>harq</i>	HARQ handle
in	<i>params</i>	Decode parameters
out	<i>output</i>	Decoded output buffer
out	<i>output_len</i>	Actual decoded length

Returns

[k_rx_ok](#) on successful decode
[k_rx_err_protocol_error](#) if decode failed (more retries possible)
[k_rx_err_invalid_arg](#) if any pointer is nullptr
[k_rx_err_invalid_state](#) if not initialized

Definition at line 614 of file [rx_harq.c](#).

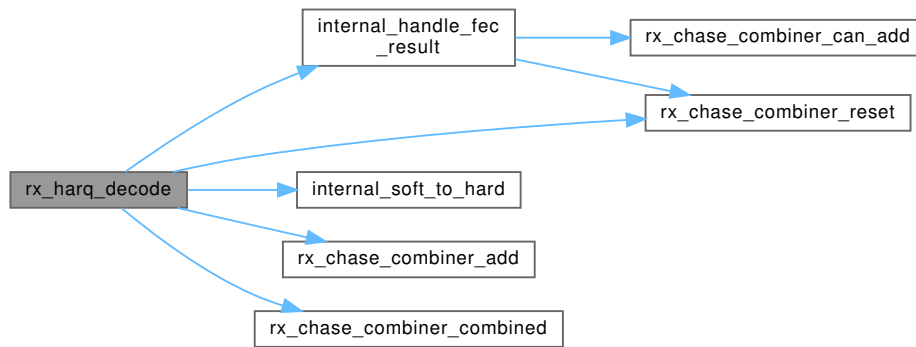
```

00618 {
00619     if (harq == nullptr || params == nullptr || output == nullptr || output_len == nullptr) {
00620         return k_rx_err_invalid_arg;
00621     }
00622     if (harq->initialized == k_harq_false) {
00623         return k_rx_err_invalid_state;
00624     }
00625     if (params->soft_bits == nullptr || params->soft_len == 0) {
00626         return k_rx_err_invalid_arg;
00627     }
00628
00629     /* Add soft bits to combiner */
00630     rx_err_t err = rx_chase_combiner_add(&harq->combiner, params->soft_bits, params->soft_len);
00631     if (err != k_rx_ok && err != k_rx_err_busy) {
00632         return err;
00633     }
00634
00635     /* Get combined soft bits into handle's buffer (thread-safe) */
00636     uint32_t combined_len = 0U;
00637     err = rx_chase_combiner_combined(&harq->combiner, harq->decode_buffer, &combined_len);
00638     if (err != k_rx_ok) {
00639         return err;
00640     }
00641
00642     /* Non-FEC path: direct soft-to-hard conversion */
00643     if (harq->fec_enabled == k_harq_false) {
00644         err = internal_soft_to_hard(harq->decode_buffer,
00645                                   combined_len,
00646                                   params->expected_output_len,
00647                                   output,
00648                                   output_len);
00649         if (err != k_rx_ok) {
00650             return err;
00651         }
00652         harq->retry_count++;
00653         /* Ignore return - combiner is already validated via harq->initialized */
00654         (void)rx_chase_combiner_reset(&harq->combiner);
00655         harq->state = k_harq_state_idle;
00656         return k_rx_ok;
00657     }
00658
00659     /* FEC path: decode combined soft bits */
00660     const rx_fec_decode_soft_params_t decode_params = {
00661         .soft_bits      = harq->decode_buffer,
00662         .soft_len       = combined_len,
00663         .expected_output_len = params->expected_output_len,
00664         .output         = output,
00665         .output_len     = output_len,
00666     };
00667
00668     err = rx_fec_decode_soft(&harq->decoder, &decode_params);
00669
00670     return internal_handle_fec_result(harq, err);
00671 }

```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decode_buffer](#), [rx_harq_handle_t::decoder](#), [rx_harq_decode_params_t::](#)
[rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [internal_handle_fec_result\(\)](#), [internal_soft_to_hard\(\)](#),
[k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_add\(\)](#), [rx_chase_combiner_combined\(\)](#),
[rx_chase_combiner_reset\(\)](#), [rx_harq_decode_params_t::soft_bits](#), [rx_harq_decode_params_t::soft_len](#), and
[rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.3.3.12 rx_harq_deinit()

```
rx_err_t rx_harq_deinit (
    rx_harq_handle_t * harq) [nodiscard]
```

Deinitialize HARQ handle and release resources.

Deinitialize HARQ handle.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

k_rx_ok on success, k_rx_err_invalid_arg if harq is nullptr

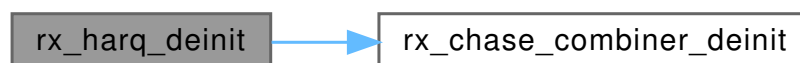
Definition at line 389 of file [rx_harq.c](#).

```

00390 {
00391     if (harq == nullptr) {
00392         return k_rx_err_invalid_arg;
00393     }
00394
00395     rx_err_t err = k_rx_ok;
00396
00397     if (harq->fec_enabled == k_harq_true) {
00398         rx_err_t fec_enc_err = rx_fec_encoder_deinit(&harq->encoder);
00399         if (fec_enc_err != k_rx_ok) {
00400             err = fec_enc_err; /* Save first error */
00401         }
00402         rx_err_t fec_dec_err = rx_fec_decoder_deinit(&harq->decoder);
00403         if (fec_dec_err != k_rx_ok && err == k_rx_ok) {
00404             err = fec_dec_err; /* Save first error */
00405         }
00406     }
00407
00408     rx_err_t combiner_err = rx_chase_combiner_deinit(&harq->combiner);
00409     if (combiner_err != k_rx_ok && err == k_rx_ok) {
00410         err = combiner_err; /* Save first error */
00411     }
00412
00413     harq->initialized = k_harq_false;
00414     return err;
00415 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_true](#), and [rx_chase_combiner_deinit\(\)](#).

Here is the call graph for this function:



5.3.3.13 rx_harq_encode()

```
rx_err_t rx_harq_encode (
    const rx_harq_handle_t * harq,
    const uint8_t * payload,
    uint32_t payload_len,
    uint8_t * output,
    uint32_t output_size,
    uint32_t * output_len) [nodiscard]
```

Encode payload with FEC (if enabled).

Parameters

in	<i>harq</i>	HARQ handle
in	<i>payload</i>	Input payload
in	<i>payload_len</i>	Payload length
out	<i>output</i>	Output buffer (must be large enough for encoded data)
in	<i>output_size</i>	Size of output buffer in bytes
out	<i>output_len</i>	Actual output length

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if any pointer is nullptr
 k_rx_err_invalid_size if output buffer too small
 k_rx_err_invalid_state if not initialized

Definition at line 462 of file rx_harq.c.

```
00468 {
00469     if (harq == nullptr || payload == nullptr || output == nullptr || output_len == nullptr) {
00470         return k_rx_err_invalid_arg;
00471     }
00472     if (harq->initialized == k_harq_false) {
00473         return k_rx_err_invalid_state;
00474     }
00475 }
00476
00477 if (payload_len == 0 || payload_len > k_harq_max_payload) {
00478     return k_rx_err_invalid_size;
00479 }
00480
00481 /* If FEC is disabled, just copy payload */
00482 if (harq->fec_enabled == k_harq_false) {
00483     /* Validate output buffer can hold payload */
00484     if (output_size < payload_len) {
00485         return k_rx_err_invalid_size;
00486     }
00487     memcpy(output, payload, payload_len);
00488     *output_len = payload_len;
00489     return k_rx_ok;
00490 }
00491
00492 /*
00493  * FEC encode: Rate 1/2 convolutional code doubles output size.
00494  * Required buffer: (payload_len * 8 + 6) * 2 bits = (payload_len * 8 + 6) / 4 bytes
00495  * Simplified worst case: payload_len * 2 + 2 bytes
00496  */
00497 const uint32_t min_output_size = (payload_len * k_harq_fec_rate_multiplier) + k_harq_tail_bytes;
00498 if (output_size < min_output_size) {
00499     return k_rx_err_invalid_size;
00500 }
00501
00502 /* FEC encode */
```

```
00503  return rx_fec_encode(&harq->encoder, payload, payload_len, output, output_len);
00504 }
```

References [rx_harq_handle_t::encoder](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_fec_rate_multiplier](#), [k_harq_max_payload](#), and [k_harq_tail_bytes](#).

5.3.3.14 rx_harq_get_retry_count()

```
uint8_t rx_harq_get_retry_count (
    const rx_harq_handle_t * harq)
```

Get number of retries attempted.

Get current retry count.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

Number of retries (0 if harq is nullptr)

Definition at line 680 of file [rx_harq.c](#).

```
00681 {
00682  if (harq == nullptr) {
00683      return 0;
00684  }
00685  return harq->retry_count;
00686 }
```

References [rx_harq_handle_t::retry_count](#).

5.3.3.15 rx_harq_get_state()

```
rx_harq_state_t rx_harq_get_state (
    const rx_harq_handle_t * harq)
```

Get current HARQ state.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

Current state, or k_harq_state_error if harq is nullptr

Definition at line 424 of file [rx_harq.c](#).

```
00425 {
00426  if (harq == nullptr) {
00427      return k_harq_state_error;
00428  }
00429  return harq->state;
00430 }
```

References [k_harq_state_error](#), and [rx_harq_handle_t::state](#).

5.3.3.16 rx_harq_init()

```
rx_err_t rx_harq_init (
    rx_harq_handle_t * harq,
    const rx_harq_config_t * config) [nodiscard]
```

Initialize HARQ handle.

Parameters

out	<i>harq</i>	Pointer to HARQ handle
in	<i>config</i>	Configuration (NULL for defaults)

Returns

k_rx_ok on success

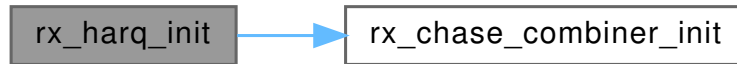
k_rx_err_invalid_arg if harq is nullptr

Definition at line 329 of file [rx_harq.c](#).

```
00330 {
00331     if (harq == nullptr) {
00332         return k_rx_err_invalid_arg;
00333     }
00334
00335     /* Initialize state */
00336     harq->state = k_harq_state_idle;
00337     harq->tx_sequence = 0;
00338     harq->rx_sequence = 0;
00339     harq->retry_count = 0;
00340
00341     /* Apply configuration */
00342     if (config != nullptr) {
00343         if (config->max_retries > k_harq_zero_retries) {
00344             harq->max_retries = config->max_retries;
00345         } else {
00346             harq->max_retries = k_harq_default_retries;
00347         }
00348         harq->fec_enabled = config->fec_enabled;
00349     } else {
00350         harq->max_retries = k_harq_default_retries;
00351         harq->fec_enabled = k_harq_true; /* FEC enabled by default */
00352     }
00353
00354     /* Initialize Chase Combiner */
00355     const bool has_combines = (config != nullptr && config->max_combines > k_harq_zero_combines);
00356     const uint8_t max_combines = has_combines ? config->max_combines : k_harq_default_combines;
00357     rx_err_t err = rx_chase_combiner_init(&harq->combiner, max_combines);
00358     if (err != k_rx_ok) {
00359         return err;
00360     }
00361
00362     /* Initialize FEC encoder if enabled */
00363     if (harq->fec_enabled == k_harq_true) {
00364         err = rx_fec_encoder_init(&harq->encoder);
00365         if (err != k_rx_ok) {
00366             return err;
00367         }
00368
00369         /* Initialize FEC decoder with survivors buffer */
00370         err =
00371             rx_fec_decoder_init(&harq->decoder, harq->decoder_survivors, k_harq_survivors_buffer_size);
00372         if (err != k_rx_ok) {
00373             (void)rx_fec_encoder_deinit(&harq->encoder); /* Cleanup, ignore errors */
00374             return err;
00375         }
00376     }
00377
00378     harq->initialized = k_harq_true;
00379     return k_rx_ok;
00380 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::decoder](#), [rx_harq_handle_t::decoder_survivors](#), [rx_harq_handle_t::encoder](#), [rx_harq_config_t::fec_enabled](#), [rx_harq_handle_t::fec_enabled](#), [rx_harq_handle_t::initialized](#), [k_harq_default_combines](#), [k_harq_default_retries](#), [k_harq_state_idle](#), [k_harq_survivors_buffer_size](#), [k_harq_true](#), [k_harq_zero_combines](#), [k_harq_zero_retries](#), [rx_harq_config_t::max_combines](#), [rx_harq_config_t::max_retries](#), [rx_harq_handle_t::max_retries](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_init\(\)](#), [rx_harq_handle_t::rx_sequence](#), [rx_harq_handle_t::state](#), and [rx_harq_handle_t::tx_sequence](#).

Here is the call graph for this function:



5.3.3.17 rx_harq_reset()

```
rx_err_t rx_harq_reset (
    rx_harq_handle_t * harq) [nodiscard]
```

Reset HARQ state for new transmission.

Reset HARQ for new transaction.

Parameters

in	<i>harq</i>	Pointer to HARQ handle
----	-------------	------------------------

Returns

`k_rx_ok` on success
`k_rx_err_invalid_arg` if `harq` is `nullptr`
`k_rx_err_invalid_state` if not initialized

Definition at line 441 of file [rx_harq.c](#).

```

00442 {
00443     if (harq == nullptr) {
00444         return k_rx_err_invalid_arg;
00445     }
00446
00447     if (harq->initialized == k_harq_false) {
00448         return k_rx_err_invalid_state;
00449     }
00450
00451     harq->state = k_harq_state_idle;
00452     harq->retry_count = 0;
00453
00454     const rx_err_t err = rx_chase_combiner_reset(&harq->combiner);
00455     if (err != k_rx_ok) {
00456         return err;
00457     }
00458
00459     return k_rx_ok;
00460 }
```

References [rx_harq_handle_t::combiner](#), [rx_harq_handle_t::initialized](#), [k_harq_false](#), [k_harq_state_idle](#), [rx_harq_handle_t::retry_count](#), [rx_chase_combiner_reset\(\)](#), and [rx_harq_handle_t::state](#).

Here is the call graph for this function:



5.4 rx_harq.c

[Go to the documentation of this file.](#)

```

00001 /* lib/rx_harq/src/rx_harq.c */
00002
00003 /**
00004  * @file rx_harq.c
00005  * @brief Hybrid Automatic Repeat Request (HARQ) Protocol Implementation
00006  *
00007  * @details
00008  * Implements Chase Combining HARQ Type I combining FEC with soft bit accumulation
00009  * across retransmissions. This is a C port of star-gateway/internal/harq/ and
00010  * fec/combiner.go for bit-exact compatibility between RX72N and RPi5.
00011  *
00012  * ## Implementation Architecture
00013  *
00014  * ### Three-Layer Design
00015  *
00016  * 1. **Chase Combiner** (rx_chase_combiner_t)
00017  *   - Accumulates soft bits element-wise across transmissions
00018  *   - Uses int16 accumulators (prevents overflow with <=255 combines)
00019  *   - Clamps output to [-127, +127] range before decoding
00020  *
00021  * 2. **FEC Codec** (rx_fec_encoder_t, rx_fec_decoder_t)
00022  *   - Optional layer for error correction
00023  *   - K=7, rate 1/2 convolutional code
00024  *   - Integrated seamlessly with combiner
00025  *
00026  * 3. **HARQ State Machine** (rx_harq_handle_t)
00027  *   - Manages retransmission protocol
00028  *   - Tracks retry count and sequence numbers
00029  *   - Coordinates encoder, decoder, and combiner
00030  *
00031  * ## Chase Combining Algorithm
00032  *
00033  * For @f$ n @f$ transmissions, accumulate soft bits:
00034  *
00035  * @f[
00036  * \text{acc}[i] = \sum_{k=1}^n s_k[i]
00037  * @f]
00038  *
00039  * Then clamp to valid range:
00040  *
00041  * @f[
00042  * s_{\text{combined}}[i] = \begin{cases}
00043  * +127 & \text{if } \text{acc}[i] > 127 \\
00044  * -127 & \text{if } \text{acc}[i] < -127 \\
00045  * \text{acc}[i] & \text{otherwise}
00046  * \end{cases}
00047  * @f]
00048  *
00049  * ## Overflow Prevention
00050  *
00051  * Accumulators use int16 (range +/-32768):
00052  * - Max combines: 255
00053  * - Max soft bit: +/-127
00054  * - Max accumulator value: +/- (255 x 127) = +/-32385
00055  * - **Safe**: 32385 < 32768 (no overflow with default max_combines=3)
00056  *
00057  * ## Protocol Flow
00058  *
00059  * @dot
00060  * digraph harq_flow {
00061  *   rankdir=TB;
00062  *   node [shape=box, style=rounded];
00063  *
00064  *   tx [label="Transmit\nFEC-encoded data"];
00065  *   rx [label="Receive\nnoisy soft bits"];
00066  *   add [label="Add to\nCombiner"];
00067  *   decode [label="Decode\ncombined bits"];
00068  *   success [label="Success\nSend ACK", fillcolor=green];
00069  *   retry [label="Failed\nretry_count++"];
00070  *   max [label="Max retries?\nSend NACK", shape=diamond];
00071  *   error [label="Error\nGive up", fillcolor=red];
00072  *
00073  *   tx -> rx -> add -> decode;
00074  *   decode -> success [label="OK"];
00075  *   decode -> max [label="FAIL"];
00076  *   max -> retry [label="No"];
00077  *   retry -> tx;
00078  *   max -> error [label="Yes"];
00079  * }
00080  * @enddot
00081  *
00082  * @par Module Dependencies

```

```

00083 * - [rx_harq.h](rx_harq.h.html): Public API
00084 * - [rx_fec.h](rx_fec.h.html): FEC codec
00085 * - [rx_bit_constants.h](rx_bit_constants.h.html): Bit manipulation
00086 * - string.h: memset, memcpy
00087 *
00088 * @par NASA Power of 10 Compliance
00089 * - **Rule 1**: [OK] No recursion, goto, setjmp/longjmp
00090 * - **Rule 2**: [OK] Loops bounded by k_harq_soft_buffer_size
00091 * - **Rule 3**: [OK] Zero dynamic allocation
00092 * - **Rule 4**: [OK] Functions <= 60 lines
00093 * - **Rule 5**: [OK] Extensive validation checks
00094 *
00095 * @author STAR Team
00096 * @date 2026-01-27
00097 * @copyright Copyright (c) 2026 STAR Project. Licensed under MIT License.
00098 *
00099 * @since Version 1.0.0
00100 * @version 1.0.0
00101 */
00102
00103 #include "rx_harq.h"
00104
00105 #include <string.h>
00106
00107 #include "rx_bit_constants.h"
00108
00109 /** @brief HARQ FEC size constants */
00110 typedef enum : uint16_t {
00111     k_harq_fec_rate_multiplier = 2, /**< Rate 1/2: encoded length multiplier */
00112     k_harq_tail_bytes = 2, /**< Tail overhead in bytes */
00113 } harq_fec_constants_t;
00114
00115 /**
00116  * @enum harq_cfg_sentinel_t
00117  * @brief HARQ config validation sentinels -- zero values for "not configured, use default"
00118  * @details
00119  * Used to detect unconfigured (zero) values for max_combines and max_retries
00120  * fields in HARQ configuration, to fall back to project defaults.
00121  *
00122  * @invariant k_harq_zero_combines and k_harq_zero_retries equal zero, signifying
00123  * "use default config" -- these sentinels must never be changed to non-zero values.
00124  *
00125  * @code
00126  * // Checking sentinels when validating config fields:
00127  * const rx_harq_config_t config = { .max_combines = 0, .max_retries = 0 };
00128  * // max_combines == k_harq_zero_combines -> use k_harq_default_combines
00129  * // max_retries == k_harq_zero_retries -> use k_harq_default_retries
00130  * uint8_t combines = (config.max_combines > k_harq_zero_combines)
00131  *     ? config.max_combines : k_harq_default_combines;
00132  * uint8_t retries = (config.max_retries > k_harq_zero_retries)
00133  *     ? config.max_retries : k_harq_default_retries;
00134  * @endcode
00135  *
00136  * @see rx_harq_config_t HARQ configuration structure containing max_combines/max_retries
00137  * @see rx_harq_init() Applies these sentinels during initialization
00138  *
00139  * @since Version 1.0.0
00140  */
00141 typedef enum : uint8_t {
00142     k_harq_zero_combines = 0, /**< Sentinel: no max_combines configured -> fall back to default */
00143     k_harq_zero_retries = 0, /**< Sentinel: no max_retries configured -> fall back to default */
00144 } harq_cfg_sentinel_t;
00145
00146 typedef enum : uint8_t {
00147     k_harq_false = 0U,
00148     k_harq_true = 1U,
00149 } harq_bool_t;
00150
00151 /**
=====
00152  * Chase Combiner Implementation
00153  *
=====
00154  */
00155
00156 rx_err_t rx_chase_combiner_init(rx_chase_combiner_t* combiner, const uint8_t max_combines)
00157 {
00158     if (combiner == nullptr) {
00159         return k_rx_err_invalid_arg;
00160     }
00161
00162     /** Clear accumulator */
00163     memset(combiner->accumulated, 0, sizeof(combiner->accumulated));
00164
00165     combiner->expected_len = 0;
00166     combiner->count = 0;
00167     if (max_combines > k_harq_zero_combines) {

```

```

00168     combiner->max_combines = max_combines;
00169 } else {
00170     combiner->max_combines = k_harq_default_combines;
00171 }
00172 combiner->initialized = k_harq_true;
00173
00174 return k_rx_ok;
00175 }
00176
00177 rx_err_t rx_chase_combiner_deinit(rx_chase_combiner_t* combiner)
00178 {
00179     if (combiner == nullptr) {
00180         return k_rx_err_invalid_arg;
00181     }
00182
00183     combiner->initialized = k_harq_false;
00184     return k_rx_ok;
00185 }
00186
00187 rx_err_t
00188 rx_chase_combiner_add(rx_chase_combiner_t* combiner, const rx_soft_bit_t* soft_bits, uint32_t len)
00189 {
00190     if (combiner == nullptr || soft_bits == nullptr) {
00191         return k_rx_err_invalid_arg;
00192     }
00193
00194     if (combiner->initialized == k_harq_false) {
00195         return k_rx_err_invalid_state;
00196     }
00197
00198     if (len == 0) {
00199         return k_rx_err_invalid_arg;
00200     }
00201
00202     if (len > k_harq_soft_buffer_size) {
00203         return k_rx_err_invalid_size;
00204     }
00205
00206     if (combiner->count >= combiner->max_combines) {
00207         return k_rx_err_busy;
00208     }
00209
00210     /* First transmission sets the expected length */
00211     if (combiner->count == 0) {
00212         combiner->expected_len = len;
00213         /* Clear accumulator for new frame */
00214         memset(combiner->accumulated, 0, len * sizeof(int16_t));
00215     } else if (len != combiner->expected_len) {
00216         return k_rx_err_invalid_size;
00217     }
00218
00219     /*
00220      * Element-wise addition (accumulate).
00221      *
00222      * Loop bound verification (NASA Rule 2):
00223      * - len is validated: len > 0 (line 62-64) AND len <= k_harq_soft_buffer_size (line 66-68)
00224      * - Maximum iterations: k_harq_soft_buffer_size (16400)
00225      * - Array bounds: accumulated[i] and soft_bits[i] are safe for i < len
00226      */
00227     for (uint32_t i = 0; i < len; i++) {
00228         combiner->accumulated[i] += (int16_t)soft_bits[i];
00229     }
00230
00231     combiner->count++;
00232     return k_rx_ok;
00233 }
00234
00235 rx_err_t rx_chase_combiner_combined(const rx_chase_combiner_t* combiner,
00236                                     rx_soft_bit_t* output,
00237                                     uint32_t* len)
00238 {
00239     if (combiner == nullptr || output == nullptr || len == nullptr) {
00240         return k_rx_err_invalid_arg;
00241     }
00242
00243     if (combiner->initialized == k_harq_false) {
00244         return k_rx_err_invalid_state;
00245     }
00246
00247     if (combiner->count == 0 || combiner->expected_len == 0) {
00248         return k_rx_err_invalid_state;
00249     }
00250
00251     /*
00252      * Clamp accumulated values to SoftBit range [-127, +127].
00253      *
00254      * Loop bound verification (NASA Rule 2):

```

```

00255  * - expected_len is set only via rx_chase_combiner_add() with validated len
00256  * - expected_len > 0 (validated at line 103-105)
00257  * - expected_len <= k_harq_soft_buffer_size (validated when set in add)
00258  * - Maximum iterations: k_harq_soft_buffer_size (16400)
00259  * - Array bounds: accumulated[i] and output[i] are safe for i < expected_len
00260  */
00261  for (uint32_t i = 0; i < combiner->expected_len; i++) {
00262      const int16_t acc = combiner->accumulated[i];
00263      if (acc > k_soft_bit_max) {
00264          output[i] = k_soft_bit_max;
00265      } else if (acc < k_soft_bit_min) {
00266          output[i] = k_soft_bit_min;
00267      } else {
00268          output[i] = (rx_soft_bit_t)acc;
00269      }
00270  }
00271
00272  *len = combiner->expected_len;
00273  return k_rx_ok;
00274 }
00275
00276 rx_err_t rx_chase_combiner_reset(rx_chase_combiner_t* combiner)
00277 {
00278     if (combiner == nullptr) {
00279         return k_rx_err_invalid_arg;
00280     }
00281
00282     if (combiner->initialized == k_harq_false) {
00283         return k_rx_err_invalid_state;
00284     }
00285
00286     /* Clear the entire accumulator to prevent stale data */
00287     memset(combiner->accumulated, 0, sizeof(combiner->accumulated));
00288     combiner->count = 0;
00289     combiner->expected_len = 0;
00290
00291     return k_rx_ok;
00292 }
00293
00294 /**
00295  * @brief Check if combiner can accept more transmissions
00296  *
00297  * @param[in] combiner Pointer to Chase combiner instance
00298  *
00299  * @return true if more transmissions can be added, false otherwise
00300  */
00301 bool rx_chase_combiner_can_add(const rx_chase_combiner_t* combiner)
00302 {
00303     if (combiner == nullptr || combiner->initialized == k_harq_false) {
00304         return false;
00305     }
00306     return combiner->count < combiner->max_combines;
00307 }
00308
00309 /**
00310  * @brief Get number of transmissions combined so far
00311  *
00312  * @param[in] combiner Pointer to Chase combiner instance
00313  *
00314  * @return Number of combined transmissions (0 if combiner is nullptr)
00315  */
00316 uint8_t rx_chase_combiner_count(const rx_chase_combiner_t* combiner)
00317 {
00318     if (combiner == nullptr) {
00319         return 0;
00320     }
00321     return combiner->count;
00322 }
00323
00324 /*
=====
00325  * HARQ Handle Implementation
00326  *
=====
00327  */
00328
00329 rx_err_t rx_harq_init(rx_harq_handle_t* harq, const rx_harq_config_t* config)
00330 {
00331     if (harq == nullptr) {
00332         return k_rx_err_invalid_arg;
00333     }
00334
00335     /* Initialize state */
00336     harq->state = k_harq_state_idle;
00337     harq->tx_sequence = 0;
00338     harq->rx_sequence = 0;
00339     harq->retry_count = 0;

```

```

00340
00341 /* Apply configuration */
00342 if (config != nullptr) {
00343     if (config->max_retries > k_harq_zero_retries) {
00344         harq->max_retries = config->max_retries;
00345     } else {
00346         harq->max_retries = k_harq_default_retries;
00347     }
00348     harq->fec_enabled = config->fec_enabled;
00349 } else {
00350     harq->max_retries = k_harq_default_retries;
00351     harq->fec_enabled = k_harq_true; /* FEC enabled by default */
00352 }
00353
00354 /* Initialize Chase Combiner */
00355 const bool has_combines = (config != nullptr && config->max_combines > k_harq_zero_combines);
00356 const uint8_t max_combines = has_combines ? config->max_combines : k_harq_default_combines;
00357 rx_err_t err = rx_chase_combiner_init(&harq->combiner, max_combines);
00358 if (err != k_rx_ok) {
00359     return err;
00360 }
00361
00362 /* Initialize FEC encoder if enabled */
00363 if (harq->fec_enabled == k_harq_true) {
00364     err = rx_fec_encoder_init(&harq->encoder);
00365     if (err != k_rx_ok) {
00366         return err;
00367     }
00368
00369 /* Initialize FEC decoder with survivors buffer */
00370 err =
00371     rx_fec_decoder_init(&harq->decoder, harq->decoder_survivors, k_harq_survivors_buffer_size);
00372 if (err != k_rx_ok) {
00373     (void)rx_fec_encoder_deinit(&harq->encoder); /* Cleanup, ignore errors */
00374     return err;
00375 }
00376 }
00377
00378 harq->initialized = k_harq_true;
00379 return k_rx_ok;
00380 }
00381
00382 /**
00383  * @brief Deinitialize HARQ handle and release resources
00384  *
00385  * @param[in] harq Pointer to HARQ handle
00386  *
00387  * @return k_rx_ok on success, k_rx_err_invalid_arg if harq is nullptr
00388  */
00389 rx_err_t rx_harq_deinit(rx_harq_handle_t* harq)
00390 {
00391     if (harq == nullptr) {
00392         return k_rx_err_invalid_arg;
00393     }
00394
00395     rx_err_t err = k_rx_ok;
00396
00397     if (harq->fec_enabled == k_harq_true) {
00398         rx_err_t fec_enc_err = rx_fec_encoder_deinit(&harq->encoder);
00399         if (fec_enc_err != k_rx_ok) {
00400             err = fec_enc_err; /* Save first error */
00401         }
00402         rx_err_t fec_dec_err = rx_fec_decoder_deinit(&harq->decoder);
00403         if (fec_dec_err != k_rx_ok && err == k_rx_ok) {
00404             err = fec_dec_err; /* Save first error */
00405         }
00406     }
00407
00408     rx_err_t combiner_err = rx_chase_combiner_deinit(&harq->combiner);
00409     if (combiner_err != k_rx_ok && err == k_rx_ok) {
00410         err = combiner_err; /* Save first error */
00411     }
00412
00413     harq->initialized = k_harq_false;
00414     return err;
00415 }
00416
00417 /**
00418  * @brief Get current HARQ state
00419  *
00420  * @param[in] harq Pointer to HARQ handle
00421  *
00422  * @return Current state, or k_harq_state_error if harq is nullptr
00423  */
00424 rx_harq_state_t rx_harq_get_state(const rx_harq_handle_t* harq)
00425 {
00426     if (harq == nullptr) {

```

```

00427     return k_harq_state_error;
00428 }
00429 return harq->state;
00430 }
00431
00432 /**
00433  * @brief Reset HARQ state for new transmission
00434  *
00435  * @param[in] harq Pointer to HARQ handle
00436  *
00437  * @return k_rx_ok on success
00438  * @return k_rx_err_invalid_arg if harq is nullptr
00439  * @return k_rx_err_invalid_state if not initialized
00440  */
00441 rx_err_t rx_harq_reset(rx_harq_handle_t* harq)
00442 {
00443     if (harq == nullptr) {
00444         return k_rx_err_invalid_arg;
00445     }
00446
00447     if (harq->initialized == k_harq_false) {
00448         return k_rx_err_invalid_state;
00449     }
00450
00451     harq->state = k_harq_state_idle;
00452     harq->retry_count = 0;
00453
00454     const rx_err_t err = rx_chase_combiner_reset(&harq->combiner);
00455     if (err != k_rx_ok) {
00456         return err;
00457     }
00458
00459     return k_rx_ok;
00460 }
00461
00462 rx_err_t rx_harq_encode(const rx_harq_handle_t* harq,
00463                        const uint8_t* payload,
00464                        const uint32_t payload_len,
00465                        uint8_t* output,
00466                        const uint32_t output_size,
00467                        uint32_t* output_len)
00468 {
00469     if (harq == nullptr || payload == nullptr || output == nullptr || output_len == nullptr) {
00470         return k_rx_err_invalid_arg;
00471     }
00472
00473     if (harq->initialized == k_harq_false) {
00474         return k_rx_err_invalid_state;
00475     }
00476
00477     if (payload_len == 0 || payload_len > k_harq_max_payload) {
00478         return k_rx_err_invalid_size;
00479     }
00480
00481     /* If FEC is disabled, just copy payload */
00482     if (harq->fec_enabled == k_harq_false) {
00483         /* Validate output buffer can hold payload */
00484         if (output_size < payload_len) {
00485             return k_rx_err_invalid_size;
00486         }
00487         memcpy(output, payload, payload_len);
00488         *output_len = payload_len;
00489         return k_rx_ok;
00490     }
00491
00492     /*
00493      * FEC encode: Rate 1/2 convolutional code doubles output size.
00494      * Required buffer: (payload_len * 8 + 6) * 2 bits = (payload_len * 8 + 6) / 4 bytes
00495      * Simplified worst case: payload_len * 2 + 2 bytes
00496      */
00497     const uint32_t min_output_size = (payload_len * k_harq_fec_rate_multiplier) + k_harq_tail_bytes;
00498     if (output_size < min_output_size) {
00499         return k_rx_err_invalid_size;
00500     }
00501
00502     /* FEC encode */
00503     return rx_fec_encode(&harq->encoder, payload, payload_len, output, output_len);
00504 }
00505
00506 /**
00507  * Decode Helper Functions
00508  */
00509
00510
00511 /**

```

```

00512 * @brief Bit position constants for soft-to-hard conversion
00513 */
00514 typedef enum : uint8_t {
00515     k_msb_position = k_rx_bits_per_byte - 1U, /**< Most significant bit position */
00516     k_rounding_adjustment = k_rx_bits_per_byte - 1U, /**< Ceiling division: (n + 7) / 8 */
00517     k_soft_bit_zero_thresh = 0, /**< Threshold for soft-to-hard: >= 0 is bit 1 */
00518 } bit_constants_t;
00519
00520 /**
00521 * @brief Convert combined soft bits to hard bits (non-FEC path)
00522 *
00523 * @param[in] soft_bits Combined soft bits
00524 * @param[in] soft_bit_count Number of soft bits
00525 * @param[in] max_output_bytes Maximum output length (bytes, must be > 0)
00526 * @param[out] output Output hard bits (packed bytes)
00527 * @param[out] output_len Actual output length
00528 *
00529 * @return k_rx_ok on success
00530 * @return k_rx_err_invalid_arg if any pointer is nullptr
00531 */
00532 static rx_err_t internal_soft_to_hard(const rx_soft_bit_t* soft_bits,
00533                                     const uint32_t soft_bit_count,
00534                                     const uint32_t max_output_bytes,
00535                                     uint8_t* output,
00536                                     uint32_t* output_len)
00537 {
00538     /* Validate all parameters - Critical NULL checks */
00539     if (soft_bits == nullptr || output == nullptr || output_len == nullptr) {
00540         return k_rx_err_invalid_arg;
00541     }
00542     if (max_output_bytes == 0) {
00543         return k_rx_err_invalid_arg;
00544     }
00545
00546     /* Calculate output bytes with ceiling division */
00547     uint32_t out_bytes = (soft_bit_count + k_rounding_adjustment) / k_rx_bits_per_byte;
00548     if (out_bytes > max_output_bytes) {
00549         out_bytes = max_output_bytes;
00550     }
00551     memset(output, 0, out_bytes);
00552
00553     /*
00554     * Loop bound: i < soft_len is validated by prior check that soft_bits != nullptr.
00555     * Loop terminates when i >= soft_len OR when byte index exceeds out_bytes.
00556     * Maximum iterations: min(soft_len, out_bytes * k_rx_bits_per_byte).
00557     */
00558     for (uint32_t i = 0; i < soft_bit_count && (i / k_rx_bits_per_byte) < out_bytes; i++) {
00559         if (soft_bits[i] >= k_soft_bit_zero_thresh) {
00560             const uint32_t byte_idx = i / k_rx_bits_per_byte;
00561             const uint32_t bit_pos = k_msb_position - (i % k_rx_bits_per_byte);
00562             output[byte_idx] |= (uint8_t)(1U << bit_pos);
00563         }
00564     }
00565
00566     *output_len = out_bytes;
00567     return k_rx_ok;
00568 }
00569
00570 /**
00571 * @brief Handle FEC decode result and update HARQ state
00572 *
00573 * @param[in,out] harq HARQ handle (must not be NULL)
00574 * @param[in] result Decode result from FEC decoder
00575 *
00576 * @return k_rx_ok on success
00577 * @return k_rx_err_invalid_arg if harq is nullptr
00578 * @return k_rx_err_protocol_error if decode failed but retries available
00579 * @return result error code if max retries reached
00580 */
00581 static rx_err_t internal_handle_fec_result(rx_harq_handle_t* harq, const rx_err_t result)
00582 {
00583     /* Critical NULL check */
00584     if (harq == nullptr) {
00585         return k_rx_err_invalid_arg;
00586     }
00587
00588     harq->retry_count++;
00589
00590     if (result == k_rx_ok) {
00591         /* Ignore return - combiner is already validated via harq->initialized */
00592         (void)rx_chase_combiner_reset(&harq->combiner);
00593         harq->state = k_harq_state_idle;
00594         return k_rx_ok;
00595     }
00596
00597     /* Decode failed - check if we can retry */
00598     if (rx_chase_combiner_can_add(&harq->combiner) && harq->retry_count < harq->max_retries) {

```

```

00599     harq->state = k_harq_state_combining;
00600     return k_rx_err_protocol_error; /* Need more retransmissions */
00601 }
00602
00603 /* Max retries reached */
00604 (void)rx_chase_combiner_reset(&harq->combiner);
00605 harq->state = k_harq_state_error;
00606 return result;
00607 }
00608
00609 /*
=====
00610 * Decode API
00611 *
=====
00612 */
00613
00614 rx_err_t rx_harq_decode(rx_harq_handle_t* harq,
00615                        const rx_harq_decode_params_t* params,
00616                        uint8_t* output,
00617                        uint32_t* output_len)
00618 {
00619     if (harq == nullptr || params == nullptr || output == nullptr || output_len == nullptr) {
00620         return k_rx_err_invalid_arg;
00621     }
00622     if (harq->initialized == k_harq_false) {
00623         return k_rx_err_invalid_state;
00624     }
00625     if (params->soft_bits == nullptr || params->soft_len == 0) {
00626         return k_rx_err_invalid_arg;
00627     }
00628
00629     /* Add soft bits to combiner */
00630     rx_err_t err = rx_chase_combiner_add(&harq->combiner, params->soft_bits, params->soft_len);
00631     if (err != k_rx_ok && err != k_rx_err_busy) {
00632         return err;
00633     }
00634
00635     /* Get combined soft bits into handle's buffer (thread-safe) */
00636     uint32_t combined_len = 0U;
00637     err = rx_chase_combiner_combined(&harq->combiner, harq->decode_buffer, &combined_len);
00638     if (err != k_rx_ok) {
00639         return err;
00640     }
00641
00642     /* Non-FEC path: direct soft-to-hard conversion */
00643     if (harq->fec_enabled == k_harq_false) {
00644         err = internal_soft_to_hard(harq->decode_buffer,
00645                                   combined_len,
00646                                   params->expected_output_len,
00647                                   output,
00648                                   output_len);
00649         if (err != k_rx_ok) {
00650             return err;
00651         }
00652         harq->retry_count++;
00653         /* Ignore return - combiner is already validated via harq->initialized */
00654         (void)rx_chase_combiner_reset(&harq->combiner);
00655         harq->state = k_harq_state_idle;
00656         return k_rx_ok;
00657     }
00658
00659     /* FEC path: decode combined soft bits */
00660     const rx_fec_decode_soft_params_t decode_params = {
00661         .soft_bits = harq->decode_buffer,
00662         .soft_len = combined_len,
00663         .expected_output_len = params->expected_output_len,
00664         .output = output,
00665         .output_len = output_len,
00666     };
00667
00668     err = rx_fec_decode_soft(&harq->decoder, &decode_params);
00669
00670     return internal_handle_fec_result(harq, err);
00671 }
00672
00673 /**
00674 * @brief Get number of retries attempted
00675 *
00676 * @param[in] harq Pointer to HARQ handle
00677 *
00678 * @return Number of retries (0 if harq is nullptr)
00679 */
00680 uint8_t rx_harq_get_retry_count(const rx_harq_handle_t* harq)
00681 {
00682     if (harq == nullptr) {
00683         return 0;

```

```
00684 }
00685 return harq->retry_count;
00686 }
00687
00688 /**
00689  * @brief Check if more retries are allowed
00690  * @param[in] harq Pointer to HARQ handle
00691  * @return true if retry is possible, false otherwise
00692  */
00693 bool rx_harq_can_retry(const rx_harq_handle_t* harq)
00694 {
00695     if (harq == nullptr || harq->initialized == k_harq_false) {
00696         return false;
00697     }
00698     return (harq->retry_count < harq->max_retries) && rx_chase_combiner_can_add(&harq->combiner);
00699 }
00700
00701 }
```

